全国大学生嵌入式芯片与系统设计竞赛

FPGA 创新设计赛道　中科亿海微　选题一：基于以太网 MAC 总线实现 AFDX

# AFDX 端系统

# FPGA 设计文档

方案与具体设计

队伍名称：[待填写]

参赛成员：[待填写]

指导教师：[待填写]

目标器件：EQ6HL130，CSG484_H 封装

开发环境：eLinx 3.0.7，ModelSim SE-64 2020.4

2026 年 10 月

# 目录


**第 1 章　概述** **1**
- 1.1　设计目标 . . . 1
- 1.2　主要成果 . . . 1
- 1.3　文档结构 . . . 2

**第 2 章　需求分析与规格分解** **3**
- 2.1　赛题需求 . . . 3
- 2.2　官方答疑口径 . . . 3
- 2.3　设计规格 . . . 4
- 2.4　协议依据 . . . 5
- 2.5　追溯矩阵 . . . 5

**第 3 章　总体架构** **6**
- 3.1　系统框图 . . . 6
- 3.2　模块层次 . . . 7
- 3.3　关键设计决策 . . . 7
- 3.4　时钟与复位 . . . 8
- 3.5　时间基准 . . . 9
- 3.6　端到端数据通路 . . . 9
- 3.7　设计预算 . . . 9

**第 4 章　接口设计** **14**
- 4.1　顶层端口 . . . 14
- 4.2　GMII 接口 . . . 15
- 4.3　线路帧格式 . . . 16
- 4.4　应用接口 . . . 17
- 4.5　配置总线 . . . 17
- 4.6　事件接口 . . . 18
- 4.7　内部接口 . . . 18

**第 5 章　发送通路** **20**
- 5.1　发送核心与配置表 . . . 20
- 5.2　空闲页池 . . . 20
- 5.3　分页缓存 . . . 21
- 5.4　调度引擎 . . . 23
- 5.5　组帧器 . . . 25
- 5.6　双网发送 MAC . . . 27

**第 6 章　接收通路** **29**
- 6.1　接收核心与配置表 . . . 29
- 6.2　接收 MAC . . . 30
- 6.3　帧头解析与试写缓存 . . . 30
- 6.4　完整性检查与冗余管理 . . . 32
- 6.5　交付 . . . 34
- 6.6　统计分类 . . . 34

**第 7 章　服务功能** **36**
- 7.1　服务通道 . . . 36

7.2 RTC 时钟同步 . . . 37
7.3 SNMP 网络管理 . . . 39
7.4 615A 数据加卸载 . . . 41

第 **8** 章 配置、复位与统计 **44**
8.1 地址映射 . . . 44
8.2 配置表格式 . . . 45
8.3 读写通路 . . . 45
8.4 配置写与查表的避让 . . . 46
8.5 复位与初始化 . . . 46
8.6 统计计数器 . . . 46
8.7 异常处理汇总 . . . 47
8.8 主机编程流程 . . . 48
8.9 配置约束 . . . 49

第 **9** 章 时序与资源 **50**
9.1 时序设计规则 . . . 50
9.2 存储预算 . . . 50
9.3 综合结果 . . . 50
9.4 关键路径 . . . 51

第 **10** 章 可验证性设计 **53**
10.1 设计原则 . . . 53
10.2 可观察性 . . . 53
10.3 可控性 . . . 53

第 **11** 章 设计边界与后续工作 **54**
11.1 设计边界 . . . 54
11.2 后续工作 . . . 54

附录 **A** 源文件与默认参数 **55**
A.1 源文件 . . . 55
A.2 默认参数 . . . 55

附录 **B** 服务寄存器 **57**

附录 **C** 内部接口信号 **59**
C.1 消息接口 . . . 59
C.2 入队与授权 . . . 59
C.3 组帧器与 MAC . . . 60
C.4 帧描述与交付队列 . . . 60
C.5 服务发送总线 . . . 61

附录 **D** 术语与参考文献 **62**
D.1 术语 . . . 62
D.2 参考文献 . . . 62

# 第 1 章　概述

## 1.1　设计目标

AFDX 端系统是航电网络中主机与交换网络之间的接口：它要按每条虚拟链路（VL）的带宽分配间隔（BAG）把应用消息装成以太网帧，同时从 A、B 两路网络发出；接收时逐帧检查完整性，把两路冗余副本合并为唯一一份交付给应用。赛题要求端系统支持 ARINC 664 Part 7 通信、高完整性通信、SNMP 网络管理、ARINC 615A 数据加卸载和 RTC 时钟同步五项功能，并给出传输速率 100 Mbps、端到端时延 1–2 ms、抖动几十微秒、128 条发送 VL 和 4096 条接收 VL 的技术指标。

在 FPGA 上实现这些要求，核心矛盾在于逐链路状态的规模与逐字节线速通路之间：4096 条接收 VL 和 128 条发送 VL 的配置、序号与定时状态若放进触发器，资源随链路数线性膨胀；若放进块 RAM，每帧的查表、比较和回写又会落到关键路径上，难以在每拍一字节的数据通路下保持确定的发送时刻。

本设计实现了完整的 AFDX 端系统 `AFDX_top`，以单一 125 MHz 时钟驱动两路八位 GMII，全部由 Verilog RTL 自主实现，经 eLinx 3.0.7 综合、ModelSim 仿真验证。设计有三项关键技术：

1. 表加单主状态机。为在线速下同时支撑 128 条发送 VL 和 4096 条接收 VL，所有逐 VL 配置和状态存放在块 RAM 中，每张表只由一个状态机按固定流水读写，数据通路每拍处理一个字节。这一结构在 EQ6HL130 上只占用 24.12% 的 LUT 和 28.33% 的存储块。
2. 提前选、到点发的发送调度。为使发送时刻严格对齐 BAG，发送引擎在到期前 512 拍选出候选 VL 并预先组好帧头，到点即发。只要没有其他 VL 的帧争用线路，帧的第一个前导字节就恰在允许时刻出现，调度本身不引入抖动。
3. 自研双网 **MAC**。一个状态机驱动 A、B 两套 CRC 与输出寄存器，两网天然同拍。不经用户侧 FIFO，空闲发送延迟为 0 拍；与 eLinx 生成的三速以太网 IP 配置相比，单口 LUT 用量为 556 对 1400。

在此基础上，端系统每网每方向的线速载荷容量为 956.44 Mbps，是赛题门限的 9.5 倍；线路空闲时应用到应用的时延上界为 26.360 μs，比 1 ms 要求低近两个数量级（推导见第 3.7 节）。

## 1.2　主要成果

表 1-1 列出赛题五项功能的实现要点，表 1-2 给出技术指标的设计值。设计值均由结构参数推导，推导过程见第 3.7 节；仿真实测结果见测试报告。

表 1-1　赛题功能与实现要点

| 赛题功能 | 实现要点 | 详见 |
|---|---|---|
| ARINC 664 P7 通信 | 128 条发送 VL 按 BAG 调度，UDP/IP 封装，A、B 双网同时发送；4096 条接收 VL 按 VLID 直接查表 | 第 5、6 章 |
| 高完整性通信 | FCS、帧长、IP/UDP 头逐帧检查；每网独立的序号完整性检查；按处理顺序交付首个有效副本的冗余管理 | 第 6 章 |
| SNMP 网络管理 | v1/v2c 的 Get、GetNext、Set 和 v2 Trap，36 个管理对象，BER 逐字节编解码 | 第 7.3 节 |
| 615A 数据加卸载 | 基于 TFTP 的上传（LUI、LUS、LUR、LUH、数据文件）与介质定义卸载，8 KiB 加载区，CRC-16 校验 | 第 7.4 节 |
| RTC 时钟同步 | SNTPv4 主从对时，32 位秒加纳秒，8 ns 分辨率，无乘除法器的时间戳换算 | 第 7.2 节 |

表 1-2　技术指标与设计值

| 指标 | 选题指南要求 | 设计值 | 依据 |
|---|---|---|---|
| 传输速率 | 100 Mbps | 每网每方向载荷容量 956.44 Mbps | 每拍一字节；1471 字节满载帧连同前导、FCS 和帧间隔占 1538 拍（第 3.7 节） |
| 端到端时延 | <1–2 ms | 线路空闲时 ≤26.360 µs | 发送 ≤100 拍、线路 1525 拍、接收首字节 ≤200 拍、逐字节交付 $n-1$ 拍；线路争用另加式 (3-3) 的等待 |
| 发送抖动 | ≤ 几十微秒 | 无线路争用时为 0；争用时不超过前面帧的占线、提前授权等待与扫描开销之和，满足配置约束时不超过 50 µs | 提前 512 拍选出、到点起始；BAG 为 $2^k$ ms 时争用只造成一次相位后移（第 3.7.3 节） |
| 帧格式 | VLID、SN 1–255，载荷 ≤1471 B，帧长 ≤1518 B | 全部按规定生成和检查 | 帧长 $47+\max(n,17)$，64–1518 字节（第 4.3 节） |
| VL 容量 | 128 发送 / 4096 接收 | 128 / 4096，接收 VLID 任意 | 逐 VL 状态存块 RAM；65536 项 VLID 映射表 |

综合后估算方面，占用 LUT 32847、触发器 13869、CLB 4620、存储块 340，8 ns 约束下建立裕量 +2.522 ns，估算最高频率 182.548 MHz（第 9 章）。

## 1.3　文档结构

第 2 章：赛题要求和官方答疑到可验证设计规格的分解；第 3 章：总体架构与关键设计决策；第 4 章：顶层接口、线路帧格式与模块间接口；第 5、6 章：收发通路；第 7 章：RTC、SNMP、615A 三项服务；第 8 章：配置表、异常处理、主机编程流程与配置约束；第 9 章：时序设计规则与资源；第 10 章：验证观察手段；第 11 章：设计边界与后续工作。附录：源文件、参数、服务寄存器、内部接口信号表与术语和参考文献。

# 第 2 章　需求分析与规格分解

本章把选题指南的功能、指标和接口要求，连同官方答疑的口径，分解为编号的设计规格，并给出规格到设计章节和测试用例的追溯关系。下文以 R$n$ 表示官方《FPGA 创新大赛问题汇总》（1010 版）Sheet1 的第 $n$ 行。

## 2.1　赛题需求

选题指南（正文第 2–4 页）对 AFDX 端系统提出三类要求：

- 功能：ARINC 664 Part 7 通信、高完整性数据通信、SNMP 网络管理、615A 数据加卸载、RTC 时钟同步。
- 技术指标：传输速率 100 Mbps；帧格式基于 IEEE 802.3 加 ARINC 664 扩展，含 VLID 和 1–255 的序列号，有效载荷不超过 1471 字节，总帧长不超过 1518 字节；端到端时延小于 1–2 ms，抖动不超过几十微秒；最多 128 条发送 VL、4096 条接收 VL。
- 接口与编码：顶层模块统一采用 `AFDX_top` 模板的 8 个端口；端口、信号和寄存器用小写，参数用大写，重要代码段简洁注释。

## 2.2　官方答疑口径

表 2-1 汇总与本设计相关的答疑及本设计的处理方式。

表 2-1　官方答疑与本设计的处理

| 答疑 | 要点 | 本设计的处理 |
|---|---|---|
| R3、R61 | MAC 接口为 GMII，即 PHY 接口；MII 可能达不到吞吐 | 顶层 8 个端口承载 A 网八位 GMII 字节流，含前导和 FCS |
| R30 | 不绑定器件型号 | 选用 eLinx 中的 EQ6HL130，CSG484_H 封装 |
| R46 | MAC 可调用 eLinx IP | 自研双网 MAC（第 5.6 节） |
| R47、R48 | 不规定协议版本，参考协议规范 | 按表 2-3 所列规范实现 |
| R49 | 可酌情增加接口 | 新增 B 网、应用、配置和事件端口（第 4 章） |
| R50、R62 | ready 信号可不用；valid 即本拍字节有效 | `rx_ready` 不使用；`tx_ready` 表示初始化完成 |
| R58 | 吞吐率达到 100 Mbps | 以去重后有效载荷计，设计容量 956.44 Mbps（第 3.7 节） |
| R59 | 时钟频率与接口速率匹配 | 125 MHz 时钟，GMII 每拍一字节 |
| R60 | GMII 接口时延不超过 2 ms | 以应用口到应用口计时，包含收发两侧全部处理；线路空闲上界 26.360 µs |

关于 R60 的计时边界：本设计从应用提交消息末字节量到对端应用交付末字节，只扣除测试设定的线路延迟。这一区间包含发送排队、组帧、MAC、线路串行和接收检查的全部时间，覆盖了 GMII 到 GMII 的区间，因此是更严格的口径。

## 2.3　设计规格

表 2-2 把上述需求分解为编号规格。F 为功能规格，P 为性能规格，I 为接口规格，C 为实现约束。

表 2-2　设计规格

| 编号 | 规格 | 内容 |
|---|---|---|
| F1 | 用户数据发送 | 每端 128 条发送 VL，每条独立配置 BAG（以时钟周期计，不超过 $2^{30}$）、VLID、源/目的 IP 与端口、最大载荷和队列深度；消息 1–1471 字节；按 VL 配置从 A、B 一网或两网发送 |
| F2 | 用户数据接收 | 4096 条接收 VL，任意 16 位 VLID 可映射；检查前导、FCS、帧长、IP 头校验和、UDP 长度与校验和；每网独立的完整性检查；启用冗余管理且副本到达间隔在 `skew_max` 内时，同一消息只交付一次 |
| F3 | SNMP 网络管理 | SNMPv1/v2c 的 Get、GetNext、Set 和 SNMPv2-Trap；36 个管理对象；community 校验 |
| F4 | 615A 数据加卸载 | 基于 TFTP 的上传操作和介质定义卸载；LUH 头文件与数据文件 CRC-16 校验；超时重传 |
| F5 | RTC 时钟同步 | SNTPv4 主从对时；32 位秒加 32 位纳秒；回应须回显本次请求的 Transmit 时间戳 |
| P1 | 吞吐 | 每方向有效载荷不低于 100 Mbps |
| P2 | 端到端时延 | 不超过 2000 µs |
| P3 | 发送抖动 | 不超过 50 µs（“几十微秒”的检查取值） |
| P4 | 帧格式 | 帧长 64–1518 字节（含 FCS）；SN 复位后首帧为 0，之后 1–255 循环 |
| P5 | 容量 | 128 条发送 VL、4096 条接收 VL 全部可配置并可同时启用 |
| I1 | 官方端口 | `AFDX_top` 的 8 个端口名称、方向和位宽不变 |
| I2 | 线路接口 | A、B 两路八位 GMII，每拍一字节，帧间隔不少于 12 拍 |
| I3 | 扩展接口 | 应用发送/接收、配置总线、统计事件端口 |
| C1 | 时钟 | 单一 125 MHz 时钟，8 ns 约束下综合建立裕量不小于 2.0 ns |
| C2 | 编码规范 | 信号小写、参数大写；每个模块开头说明作用、接口和关键时序 |

## 2.4　协议依据

表 2-3　协议依据与版本

| 功能 | 依据 | 实现范围 |
|---|---|---|
| AFDX 网络 | ARINC 664 Part 7（2005-06-27） | 帧格式、VL 与 BAG、完整性检查、冗余管理 |
| 数据加卸载 | ARINC 615A-2（2002-05-10）、ARINC 665-1、RFC 1350 | 上传、介质定义卸载；协议文件版本 A4，兼容 A3 |
| 网络管理 | RFC 1157、RFC 3416、RFC 3418 | SNMPv1/v2c，BER 编码 |
| 时钟同步 | RFC 4330 | SNTPv4 48 字节报文 |

## 2.5　追溯矩阵

表 2-4 给出每条规格对应的设计章节和验证用例，测试用例的结果见测试报告。

表 2-4　规格追溯矩阵

| 规格 | 设计章节 | 验证用例 |
|---|---|---|
| F1 | 5.3–5.6 | `tb_tx_basic`、`tb_tx_bag`、`tb_tx_multi`、`tb_tx_drop`、`tb_sys_load` |
| F2 | 6.2–6.5 | `tb_rx_basic`、`tb_rx_fault`、`tb_rx_icrm`、`tb_rm_phase`、`tb_sys_fault` |
| F3 | 7.3 | `tb_sys_snmp`、`tb_sys_snmp_settrap`、`tb_judge_snmp` |
| F4 | 7.4 | `tb_sys_615a`、`tb_net_services` |
| F5 | 7.2 | `tb_sys_rtc`、`tb_rtc_recovery`、`tb_rtc_originate` |
| P1–P3 | 3.7、5.4 | `tb_sys_perf`、`tb_gbe_burst` |
| P4 | 4.3 | `tb_mac_tx`、`tb_mac_rx`、`tb_rx_icrm` |
| P5 | 5.1、6.1 | `tb_judge_capacity`、`tb_rx_load` |
| I1–I3 | 第 4 章 | `tb_sys_net`、`tb_sys_cfg`、`tb_elinx` |
| C1、C2 | 第 9 章 | 综合时序报告、网表仿真 |

# 第 3 章　总体架构

## 3.1　系统框图

图 3-1 是 AFDX_top 的系统框图。端系统由八个一级模块组成：发送核心和接收核心构成线速数据通路；服务通道把应用口与 RTC、SNMP、615A 三个服务接到同一对收发核心上；配置模块提供寄存器、统计计数器和三张配置表的写入口；时间基准为全设计提供自由运行的周期计数。各模块的分工见表 3-1。

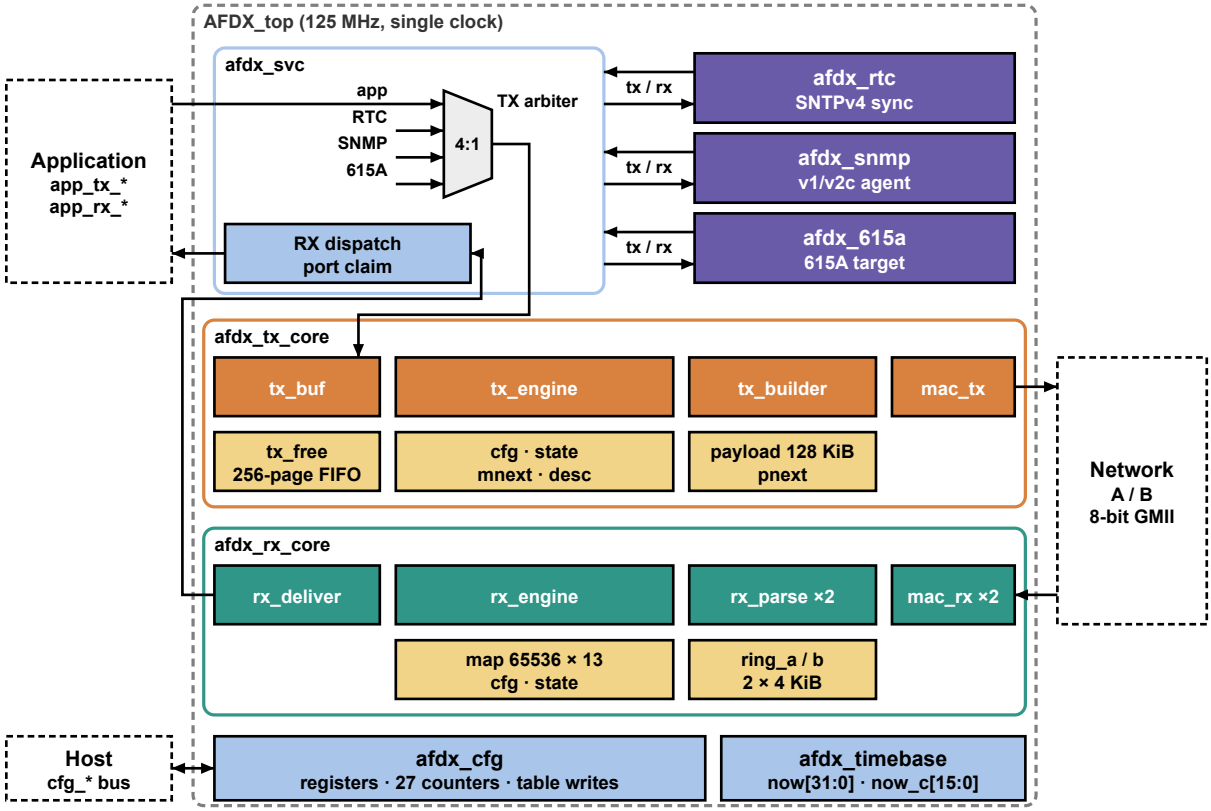



图 3-1　AFDX_top 系统框图。橙、绿、紫、黄、蓝色分别标识发送通路、接收通路、服务、块 RAM 与配置控制，虚线框为片外对象；表块放在主要访问它的模块下方，表名省略 tx_、rx_ 前缀；省略配置写入与统计脉冲连线

表 3-1　一级模块分工

| 模块 | 职责 |
|---|---|
| `afdx_tx_core` | 发送核心：消息分页缓存、128 条 VL 的 BAG 调度、组帧、双网 MAC（第 5 章） |
| `afdx_rx_core` | 接收核心：双网 MAC、帧头解析、完整性检查、冗余管理、交付（第 6 章） |
| `afdx_svc` | 服务通道：应用与三个服务按整条消息轮询发送；按 UDP 目的端口把接收消息分给服务或应用 |
| `afdx_rtc` | SNTPv4 主从时钟同步（第 7.2 节） |
| `afdx_snmp` | SNMP 代理（第 7.3 节） |
| `afdx_615a` | 615A 加卸载目标（第 7.4 节） |
| `afdx_cfg` | 地址译码、寄存器、27 个统计计数器、配置表写入握手（第 8 章） |
| `afdx_timebase` | 32 位周期计数 `now` 与粗计时 `now_c` |

## 3.2　模块层次

`AFDX_top` 例化八个一级模块；发送核心和接收核心各自例化其子模块和全部配置表。所有 RAM 都通过同一个简单双口 RAM 模块 `afdx_ram_sdp` 例化，共 19 个实例。层次如下：

```
AFDX_top
 +- afdx_timebase
 +- afdx_cfg
 +- afdx_svc
 +- afdx_tx_core
 |   +- afdx_tx_buf      +- afdx_tx_free
 |   +- afdx_tx_engine   +- afdx_tx_builder
 |   +- afdx_mac_tx      +- afdx_crc32_d8 x2
 |   +- afdx_ram_sdp x7: tx_cfg tx_state tx_desc tx_mnext
 |                       tx_pnext tx_payload tx_free
 +- afdx_rx_core
 |   +- afdx_mac_rx x2   +- afdx_rx_parse x2
 |   +- afdx_rx_engine   +- afdx_rx_deliver
 |   +- afdx_ram_sdp x5: rx_map rx_cfg rx_state rx_ring_a rx_ring_b
 +- afdx_rtc
 |   +- afdx_ntp_fraction x3
 +- afdx_snmp
 |   +- afdx_ram_sdp x2: request_ram response_ram
 +- afdx_615a
     +- afdx_ram_sdp x5: packet_rx packet_tx lur_ram header_ram load_ram
```

各源文件的作用见附录 A.1。

## 3.3　关键设计决策

表 3-2 列出影响全局的设计决策。

表 3-2 关键设计决策

| 决策 | 备选方案 | 选择理由 |
|---|---|---|
| 单一 125 MHz 时钟 | 收发分时钟、MII 25 MHz | R59 要求时钟与接口速率匹配；R61 指出 MII 可能达不到吞吐。单时钟消除跨时钟域，GMII 每拍一字节 |
| 官方 8 个端口承载真实 GMII | 端口只做握手示意 | R3 明确 MAC 接口为 GMII；端口直接输出带前导和 FCS 的线路字节，便于评测 testbench 直接对接 |
| 自研 MAC | 调用 eLinx 三速以太网 IP | R46 允许调用 IP 但不强制。自研 MAC 单口 LUT 556、空闲发送延迟 0 拍，且 A、B 两网共用一个状态机天然同拍（第 5.6 节） |
| 逐 VL 状态存块 RAM、单主状态机 | 逐 VL 寄存器阵列、并行比较 | 4096 条接收 VL 用寄存器会使资源随 VL 数线性增长；每张表只归一个状态机，读写冲突在结构上消除 |
| 提前选、到点发 | 到期后再扫描 | 128 条 VL 扫描一遍 256 拍，长于最短帧时隙 84 拍；提前 512 拍选出并预备帧头，发送时刻精确对齐 BAG |
| 接收先试写后提交 | 整帧缓存后再检查 | 不需要整帧缓存，检查失败一步回退，坏帧不占缓存 |
| 服务按消息轮询共享发送核心 | 每个服务独立的 MAC 或 VL 通道 | 服务报文与用户报文遵守同一 BAG 调度；无服务报文时应用直通，零附加延迟 |
| 三项服务用硬件状态机实现 | 软核处理器加协议软件 | EQ6HL130 没有硬核处理器，软核需另占程序存储和总线，且 RTC 的时间戳必须在报文握手那一拍由硬件采样；状态机方案的时序在 RTL 中逐拍确定，与收发核心用同一套设计规则和验证平台。代价是三项服务合计约 16.2k 个逻辑单元，与收发核心（约 14.5k）相当，其中 615A 占比最大，约 7.2k LC（第 9 章） |

## 3.4 时钟与复位

全设计只有 `clk` 一个时钟，频率 125 MHz，周期 8 ns，所有寄存器在其上升沿更新，没有跨时钟域路径。`reset` 为高有效的同步复位，设计内部不再同步，要求外部复位的释放与 `clk` 同步；上板时由板级复位同步器提供。

单时钟的前提是 GMII 收发数据都与 `clk` 同步。赛题以单一时钟下的 GMII 字节流定义接口（R3、R59），仿真平台满足这一点。实体 PHY 的接收数据随 PHY 恢复的 RX_CLK 变化，发送需要输出 GTX_CLK；上板时须在接收侧加一个以帧为单位的弹性缓冲，把 RX_CLK 域的字节流转到 `clk` 域，发送侧由 `clk` 经输出寄存器驱动 GTX_CLK。两者都在 GMII 引脚与收发核心之间，不改变核心设计（第 4.2 节）。

块 RAM 的内容不受复位影响，设计也不依赖其上电初值。复位结束后，发送核心清零 `tx_state`、`tx_cfg` 并装好 256 个空闲页；接收核心清零 `rx_map`、`rx_cfg` 和 `rx_state`。最长的 `rx_map` 有 65536 项，清表需 65536 拍。两个核心的 `init_done` 相与后寄存一拍成为顶层的 `tx_ready`，从解除复位到 `tx_ready` 为 1 共 65538 拍，即 524.304 μs。清表完成之前，`cfg_ready` 保持 0、配置写不被接受，第一个前导字节在此之前出现的线路帧整帧忽略，清表完成时也不会把半帧当作新帧。

## 3.5 时间基准

`afdx_timebase` 为全设计产生三个信号：

- `now[31:0]`：自由运行的周期计数，复位为 0，每拍加 1，约 34.36 s 回绕。所有早晚比较一律用“相减看符号位”，因此 BAG 不超过 $2^{30}$ 拍（约 8.59 s）。
- `now_c[15:0]`：粗计时，等于 `now[27:12]`，每 4096 拍（32.768 µs）加 1，只给接收侧冗余管理使用。
- `byte_en`：字节使能，复位期间为 0，此后恒为 1，即每拍都是字节边界。

每个时钟周期都是发送字节边界，帧起点与 BAG 不量化到 8 的倍数，BAG 可取任意整数拍（例如 100003 拍）。

## 3.6 端到端数据通路

图 3-2 给出一条用户消息从发送端应用到接收端应用的完整路径。发送侧依次经过分页缓存、调度引擎、组帧器和双网 MAC；两份帧经 A、B 网到达接收端后，各自经 MAC 和解析器检查，由接收引擎做完整性检查和冗余管理，最后由交付模块逐字节送给应用。

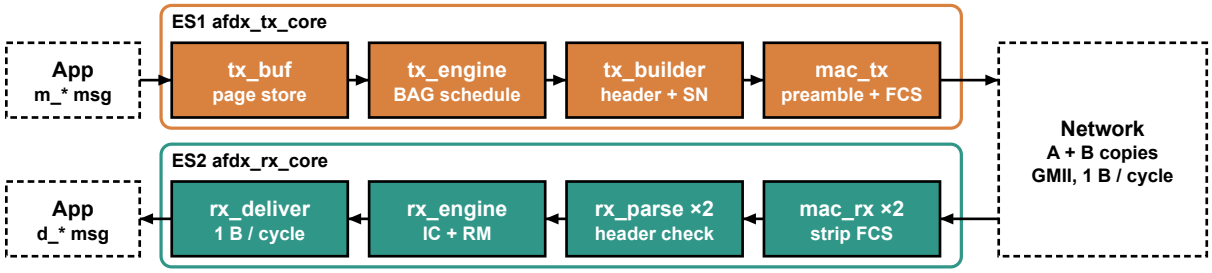



图 3-2 一条用户消息的端到端路径。上排为发送端 ES1，下排为接收端 ES2

全通路每拍处理一个字节，帧内速率 1 Gbit/s，不缓存整帧：发送侧消息先进分页缓存，到点后边读边发；接收侧边收边试写环形缓冲，检查通过即开始交付。各段的拍数和上界在第 3.7 节推导。

## 3.7 设计预算

本节由各模块的状态机拍数推导吞吐、时延、抖动、缓存和握手等待的设计值，作为第 5–8 章模块设计的约束。拍数均以 125 MHz 时钟计，1 拍 = 8 ns；仿真实测结果见测试报告。

计拍口径。本节和第 5–8 章的区段拍数，都从上一级输出有效的那一拍量到本级输出有效的那一拍。状态机从 IDLE 派发一个操作要用 1 拍，所以区段拍数比模块操作表中的操作拍数多 1：例如引擎入队区段 6 拍，等于派发 1 拍加 ENQ 操作 5 拍。

### 3.7.1 吞吐

载荷为 $n$ 字节的帧连同 8 字节前导、4 字节 FCS 和 12 拍帧间隔，在线路上占

$$C(n) = 8 + 47 + \max(n, 17) + 12 = 67 + \max(n, 17) \text{ 拍.} \tag{3-1}$$

满载帧 $n = 1471$ 占 $C = 1538$ 拍，最短帧 $n \le 17$ 占 84 拍。每网每方向的载荷容量为

$$R_{\max} = \frac{1471 \times 8 \text{ bit}}{1538 \times 8 \text{ ns}} = 956.44 \text{ Mbps.} \tag{3-2}$$

$N$ 条 VL 各以 BAG 发满载帧时，提供的载荷率为 $N \times 1471 \times 8/\text{BAG}$，可调度条件为 $N \times C \le \text{BAG}$。BAG 为 1 ms（125000 拍）时 $N \le \lfloor 125000/1538 \rfloor = 81$，此时载荷率 953.208 Mbps；77 条 VL 为 906.136 Mbps，占线 94.7%。

线速通路上的其余环节都不低于线路速率：

- 应用口收消息。`afdx_tx_buf` 每拍收一字节，每取一页停 6 拍，消息末尾入队应答停 10 拍。一条 1471 字节消息占 3 页，在应用口共用 $1471 + 3 \times 6 + 10 = 1499$ 拍，短于其线路占用 1538 拍。
- 授权与准备。引擎授权一帧用 9 拍，组帧器准备用 11 拍，合计 20 拍，在当前帧发送期间完成，小于最短帧的 84 拍时隙。
- 接收交付。交付口每拍一字节；单网到达的载荷率至多 $1471/1538$ 字节每拍，低于交付速率，环形缓冲不会持续积压。

### 3.7.2 端到端时延

时延从应用口末字节被收下的那一拍量到对端应用口交付末字节的那一拍，各区段都按两端事件所在拍号相减，并以“线路空闲、该 VL 间隔已到”为条件。按 RTL 逐拍追踪，各区段的名义拍数和设计上界见表 3-3，发送侧和接收侧的逐拍分解见图 3-3。

表 3-3 端到端时延预算（拍）

| 区段 | 名义 | 上界 | 构成 |
|---|---|---|---|
| 发送侧 | 31 | 100 | 分页缓存写描述并入队 3、引擎入队 6（派发 1 + ENQ 5）、授权 9（派发 1 + GRANT 8）、组帧准备 11（锁存 1 + 10 步）、起始判断与 MAC 2；上界另计扫描配额等待 ≤32 和一次正在进行的操作 ≤9 |
| 线路 | 1525 | 1525 | 首个前导字节到末个 FCS 字节，共 $8 + 1518$ 字节（$n = 1471$） |
| 接收至首字节 | 24 | 200 | MAC 判帧尾 3、解析器定类 3、引擎判决并登记 12、交付选队头与读环 6；上界另计另一网的帧描述 13 拍和老化让出 |
| 逐字节交付 | $n - 1$ | $n - 1$ | 首字节到末字节，每拍一字节，$n = 1471$ |
| 合计 | 3050 | 3295 | 24.400 µs / 26.360 µs |

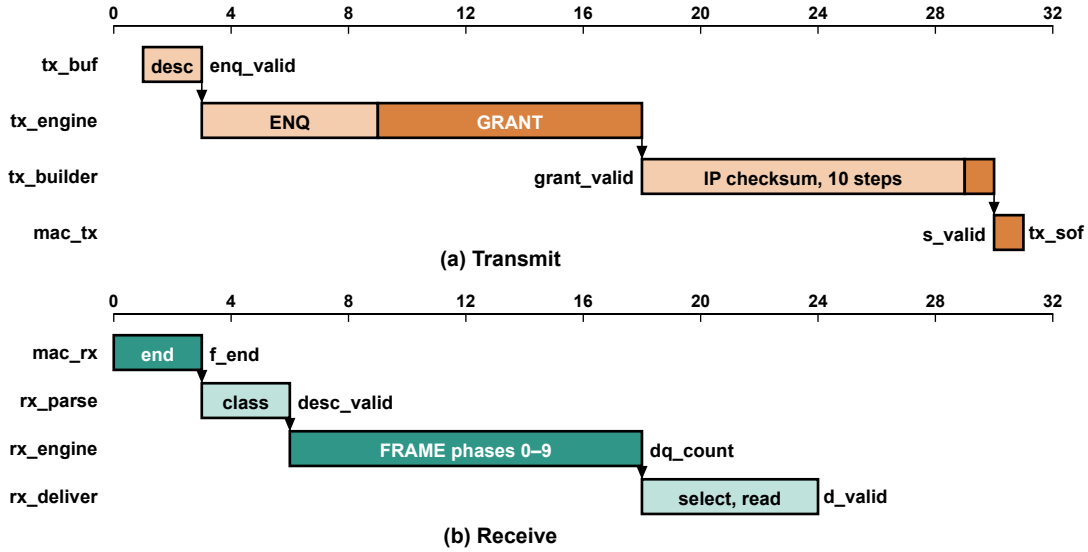



图 3-3 收发两侧的逐拍时序（线路空闲、VL 已到期）：(a) 发送侧，从末字节被接受到首个前导字节共 31 拍，入队时直接定为候选，适用于末字节在 RECV 正常收下且无需尾部分页的多字节消息；单字节或末字节跨页时多约 6 拍；(b) 接收侧，单网单帧从 GMII 最后一个 FCS 字节到首个交付字节共 24 拍

线路上有其他帧争用时，另加排在前面的帧的占线时间；端到端时延的上界由各 VL 的 BAG 和帧长配置决定，与 ARINC 664 P7 的计算方法一致。

### 3.7.3 发送抖动

记 VL $v$ 的帧允许时刻为 $T_v$（上一帧实际起始时刻加 BAG），实际起始时刻为 $S_v$，发送抖动为 $J_v = S_v - T_v$。

无争用时为 **0**。调度器能否让帧恰在 $T$ 起始，取决于候选是否在 $T$ 之前完成授权和准备。一条 VL 在 $T - 512$ 进入提前窗口，此后：

- 扫描回访。无候选时扫描每 2 拍检查一条 VL，128 条一遍 256 拍；入队每扫描 16 条至多插入一次，每次连同重扫流水约 12 拍；起始确认每帧一次，每次约 10 拍。合计任一 VL 相邻两次被检查的间隔约 400 拍。
- 授权与准备。授权 9 拍、组帧器准备 11 拍、起始判断 2 拍，共 22 拍。

$400 + 22 = 422 < 512$，留有 90 拍裕量。因此只要线路上没有其他帧，第一个前导字节恰在 $T$ 出现，调度本身不引入抖动。

争用时的上界。线路一次只发一帧，组帧器也只有一个“下一帧”位置。$v$ 到期且消息已排队时，可能因三种原因晚发：

- 占线。排在它前面的帧逐个发完，每帧占线 $C(n_u) = 67 + \max(n_u, 17)$ 拍。
- 提前授权的等待。扫描按 VL 序号轮转，候选判据是“已到期或在 512 拍内到期”，不是“最早到期”。扫描先遇到一条尚未到期的 VL $u$ 时会先授权它，组帧器持有 $u$ 等到 $T_u$ 才发，期间 $v$ 不能被授权。每个这样的 $u$ 带来的空等 $w_u < 512$ 拍。

- 扫描行程。扫描从上次选中处向后轮转，到达 $v$ 之前选中的 VL 正是排在 $v$ 前面的那些；扫描与这些帧的发送重叠进行，总行程不超过一圈，连同 $v$ 自身的授权和准备不超过约 422 拍。

记 $Q_v$ 为在 $v$ 之前发出的其他 VL 的帧，则

$$J_v \le \Big( \sum_{u \in Q_v} \big( C(n_u) + w_u \big) + 422 \Big) \times 8 \text{ ns}, \tag{3-3}$$

其中 $w_u$ 只对在自身允许时刻之前被授权的帧非零。服务报文各走自己的 VL，同样按 BAG 调度，已包含在 $Q_v$ 中。

一次争用只造成一次相位后移。下一次允许时刻按实际起始时刻加 BAG 计算（第 5.4 节）。几条 VL 相撞后，后发的 VL 相位后移，此后各自按 BAG 前进。这种错开能一直保持需要两个条件：

- 各 VL 的 BAG 互为整数倍。ARINC 664 P7 规定 BAG 取 $2^k$ ms，满足这一点；任取整数拍的 BAG 会使相位相互漂移，周期性地再次相撞。
- VL 在允许时刻之前已有消息排队。队列空时到期，`timer_active` 被清除，下一条消息到达即发，相位改由应用的提交时刻决定。

两个条件都满足时，相撞只在启动时发生，稳定后同一 VL 相邻两帧的起始间隔恰为 BAG。

满足 **50 µs** 的配置条件。按式 (3-3)，P3 的 50 µs 对应 $\sum_{u \in Q_v} (C(n_u) + w_u) \le 6250 - 422 = 5828$ 拍。例如：

- 同时到期的其他 VL 至多 3 条最长帧（$w_u = 0$）：$3 \times 1538 = 4614$ 拍，满足；
- 前面有 2 条最长帧且都被提前授权：$2 \times (1538 + 512) = 4100$ 拍，满足；
- 77 条最短帧同时到期：$76 \times 84 = 6384$ 拍，不满足。

VL 数多于上述数量时，主机应错开各 VL 首条消息的提交时刻，使任一 VL 前面可能排着的帧满足该条件；在上述两个条件下，错开只需在启动时做一次（第 8.9 节）。

与 ARINC 664 P7 的抖动公式 $40\ \mu\text{s} + \sum (20 + L_{\max}) \times 8$ bit/1 Gbps 相比，$C(n) \times 8$ ns 恰等于公式中的求和项；本设计的扫描开销 422 拍（3.4 µs）与至多 8 次提前授权等待（共 4096 拍）之和小于公式中的 40 µs 项。因此满足 P7 公式、且排在前面的提前授权帧不超过 8 个的配置，抖动不超过 P7 的 500 µs 上限。

### 3.7.4　缓存容量

- 发送载荷缓存。256 页 × 512 字节。最长消息占 $\lceil 1471/512 \rceil = 3$ 页，可同时驻留 85 条最长消息；77 条和 81 条 VL 各排一条最长消息时分别占 231 页和 243 页。页耗尽时经有界暂停后丢弃消息并计数，不会让应用无限等待。
- 接收环形缓冲。每网 4096 字节，按已提交写指针计算剩余空间，第 42 字节到来时检查能否放下本帧 $n$ 字节。帧尾到首个交付字节约 24 拍，交付速率不低于单网到

达速率，单网满载时环内驻留不超过一帧载荷加一段在途数据，4096 字节可容纳两个最长载荷。

- 交付描述队列 **rx_dq**。32 项。最坏情况是一条最长消息正在交付（1471 拍），其间单网最多到达 $\lceil 1471/84 \rceil = 18$ 个最短帧；启用冗余管理时另一网的副本被判重复，不入队，32 项足够。两网独立满速且都要交付时，单个交付口无法保证无积压，此时按溢出计数并回退。

### 3.7.5 握手等待

- 入队应答。引擎一次只做一种操作，最长的授权 9 拍；入队在扫描满 16 次或已有候选时插入，应答 ≤64 拍。
- **m_ready**。这是发送核心消息接口的 ready。取页停 6 拍；消息末尾写描述、入队应答、退页（至多 3 页，每页 3 拍）和事件共停 $2 + 64 + 9 + 1 = 76$ 拍。设计约束取 m_ready 连续为 0 不超过 96 拍。
- 应用口等待。app_tx_ready 还受服务通道仲裁控制（第 7.1 节）。三个服务都有消息时，应用至多等其各传完一条：RTC 报文 48 字节、每字节 4 拍，SNMP 回应不超过 256 字节、每字节 4 拍，615A 报文不超过 516 字节、每字节 5 拍，连同各自的取页和入队，共约 3850 拍（约 31 µs）。服务报文是低速管理流量，这种等待只在三者恰好同时有报文时出现。
- 配置写。写哪张表就等哪个核心的 cfg_ok：发送引擎最坏连续执行起始确认、入队和授权约 25 拍，接收引擎最坏处理两网各一份描述 26 拍，等待上界取 64 拍（第 8.4 节）。
- 初始化。最大的 rx_map 有 65536 项，每拍清一项，加最后一次写入和 tx_ready 的输出寄存器，共 65538 拍（524.304 µs）。

# 第 4 章　接口设计

## 4.1　顶层端口

`AFDX_top` 保留选题模板的 8 个端口，名称、方向和位宽不变，并按 R49 增加 B 网、应用、配置和事件端口。顶层共 40 个端口声明、259 位，低于 EQ6HL130 的 339 个用户 I/O。端口分组见表 4-1。

表 4-1　顶层端口

| 端口 | 方向 | 含义 |
|---|---|---|
| *选题模板端口* | | |
| `clk` | 入 | 125 MHz 时钟 |
| `reset` | 入 | 同步复位，高有效 |
| `tx_data[7:0]`、`tx_valid` | 出 | A 网 GMII 发送，相当于 TXD、TX_EN |
| `rx_data[7:0]`、`rx_valid` | 入 | A 网 GMII 接收，相当于 RXD、RX_DV |
| `tx_ready` | 出 | 收发核心初始化完成，端系统可以工作 |
| `rx_ready` | 入 | 不使用（R50、R62） |
| *冗余网络* | | |
| `tx_data_b`、`tx_valid_b`、`rx_data_b`、`rx_valid_b` | 出/入 | B 网，含义同 A 网 |
| *应用发送* | | |
| `app_tx_valid`、`app_tx_ready` | 入/出 | 字节握手 |
| `app_tx_data[7:0]`、`app_tx_last` | 入 | 消息字节；`last` 标记末字节 |
| `app_tx_vl[6:0]` | 入 | 发送 VL 序号，在首字节取样 |
| `app_tx_port_en`、`app_tx_sport[15:0]`、`app_tx_dport[15:0]` | 入 | 为 1 时用消息自带的 UDP 端口，否则用 VL 的默认端口；在首字节取样 |
| *应用接收* | | |
| `app_rx_valid`、`app_rx_data[7:0]`、`app_rx_last` | 出 | 交付字节，不带反压 |
| `app_rx_vl[11:0]`、`app_rx_sport`、`app_rx_port`、`app_rx_net` | 出 | 接收 VL 序号、UDP 源/目的端口、来自哪一网；整条消息期间不变 |
| *配置与统计* | | |
| `cfg_wr`、`cfg_rd`、`cfg_addr[17:0]`、`cfg_wdata[31:0]` | 入 | 配置总线请求 |

接下页

表 4-1 （续）

| 端口 | 方向 | 含义 |
|---|---|---|
| `cfg_rdata[31:0]`、`cfg_rvalid`、`cfg_ready` | 出 | 读数据、读有效、写就绪 |
| `ev_tx[6:0]` | 出 | 发送统计脉冲：accept、drop_dis、drop_full、drop_long、drop_nopage、tx_a、tx_b |
| `ev_a[7:0]`、`ev_b[7:0]` | 出 | 每网接收统计脉冲，见表 6-3 |
| `ev_rm_drop`、`ev_deliver` | 出 | 冗余丢弃、交付 |
| `ev_gdrop[1:0]` | 出 | 保留端口，固定为 0 |

顶层参数 `TX_PAGE_AW`、`TX_OFF_AW`、`LOOKAHEAD`、`RX_RING_AW`、`RX_DQ_AW` 的默认值分别为 8、9、512、12、5，原样传给收发核心。

## 4.2 GMII 接口

A、B 两网各有一组八位 GMII，每拍一个字节，`valid` 为 1 即表示本拍字节有效，不与 ready 握手（R62）。

- 发送：`tx_valid` 从第一个前导字节到最后一个 FCS 字节连续为 1，帧间至少空闲 12 拍，见图 4-1。GMII 输出直接来自 MAC 的输出寄存器，发送适配延迟为 0。某条 VL 未启用的那一网，整帧期间 `tx_valid` 和 `tx_data` 保持为 0。
- 接收：`rx_valid` 上升开始一帧、下降结束一帧。MAC 先寄存输入，再逐拍找帧起始符、累加 CRC 并剥离 FCS，无需缓存整帧；两帧之间只有 1 个低电平周期也能正确分开。
- 复位：复位期间输出为 0；清表完成之前开始的帧整帧忽略（第 3.4 节）。
- 时钟与错误信号：收发字节都与 `clk` 同步，接口不含 TX_CLK、RX_CLK、TX_ER、RX_ER、CRS 和 COL。全双工交换式网络没有冲突，CRS、COL 不需要；PHY 报告符号错（RX_ER）的帧，数据通常也已损坏，由 FCS 检查剔除。接实体 PHY 时的时钟处理见第 3.4 节。

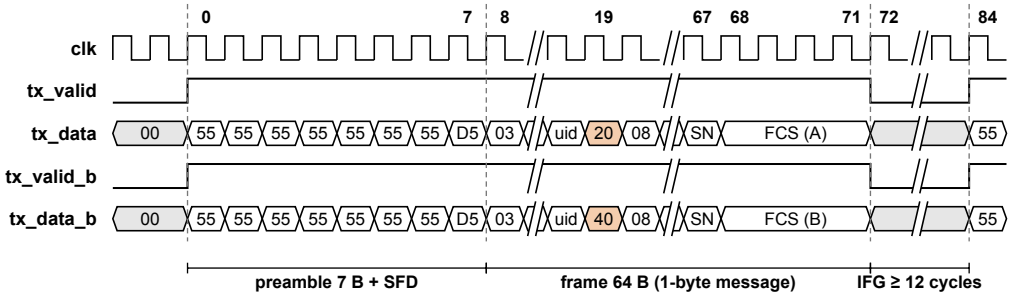



图 4-1 GMII 发送一帧的接口时序（1 字节消息，最短帧）。第 0 拍为首个前导字节，两网同拍起始，仅第 11 字节（接口号，A 为 `20h`、B 为 `40h`）和 FCS 不同；某网未启用时该网整帧保持为 0

## 4.3　线路帧格式

图 4-2 是本设计发送的 AFDX 帧。目的 MAC 和源 MAC 按 ARINC 664 Part 7 的固定格式编码：目的 MAC 为 `03 00 00 00` 加 16 位 VLID，源 MAC 为 `02 00 00` 加 16 位端系统用户 ID 和 1 字节接口号（A 网 `20`、B 网 `40`）。其后是 IPv4 头、UDP 头、应用数据、不足 17 字节时的补零，以及 1 字节序列号 SN，最后由 MAC 加 4 字节 FCS。

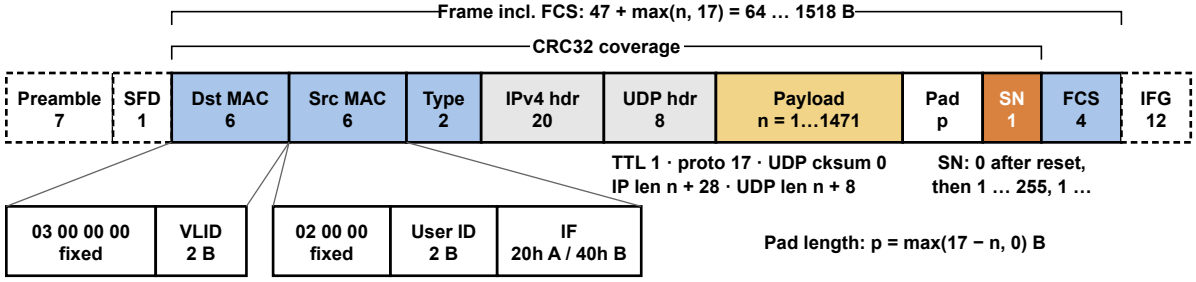



图 4-2　AFDX 帧格式。$n$ 为应用数据字节数，$1 \le n \le 1471$；虚线框为线路开销，下方展开目的 MAC 与源 MAC

表 4-2　帧字段

| 字节位置 | 长度 | 内容 |
|---|---|---|
| 0–3 | 4 | `03 00 00 00`，Part 7 规定的目的 MAC 固定部分 |
| 4–5 | 2 | VLID，高字节在前 |
| 6–8 | 3 | `02 00 00`，源 MAC 固定部分 |
| 9–10 | 2 | 端系统用户 ID（寄存器 `USER_ID`） |
| 11 | 1 | 接口号：A 网 `20`，B 网 `40` |
| 12–13 | 2 | `08 00`（IPv4） |
| 14–33 | 20 | IPv4 头：`45 00`，总长度 $n+28$，标识（每帧加 1），`00 00`，TTL `01`，协议 `11`，头校验和，源 IP，目的 IP |
| 34–41 | 8 | UDP 头：源端口，目的端口，长度 $n+8$，校验和（填 0） |
| 42 起 | $n$ | 应用数据 |
| 其后 | $17-n$ | $n<17$ 时补 0 至 17 字节 |
| 再后 | 1 | 序列号 SN |
| 最后 | 4 | FCS（CRC32，由 MAC 添加） |

由此，含 FCS 的帧长为 $47+\max(n,17)$，范围 64–1518 字节；A、B 两份帧只有第 11 字节和 FCS 不同。SN 在复位后每条 VL 的第一帧为 0，之后按 1、2、…、255、1、… 循环。组播目的 IP 按 Part 7 图 3-38 配置为 `224.224.VLID[15:8].VLID[7:0]`，发送核心直接使用配置表中的目的 IP，因此单播服务（如 615A）也能用同一通路。

CRC32 采用反射多项式 `0xEDB88320`、初值全 1、结果取反后低字节先发，覆盖从目的 MAC 到 SN 的全部字节。它实现为组合模块 `afdx_crc32_d8`（32 位当前值加 8 位数据，输出新值），收发共用。

## 4.4 应用接口

发送。应用以字节流提交消息：`app_tx_valid` 与 `app_tx_ready` 同拍为 1 时传送一个字节，`app_tx_last` 标记末字节；VL 序号和 UDP 端口在首字节取样。没有服务报文争用时，`app_tx_ready` 就是发送核心的 `m_ready`，每拍接收一个字节，仅在分配缓存页或等待入队结果的若干拍内为 0：每取一页 6 拍，消息末尾写描述并入队正常为 10 拍（图 4-3(a)），连续为 0 不超过 96 拍。服务报文正在传送时，应用要等它传完，三个服务同时有报文时约等 3850 拍（第 3.7 节）。缓存页用尽或 VL 队列已满时消息被丢弃并计数，不会让应用无限等待。

接收。一条消息的 $n$ 个字节在连续 $n$ 拍内送出，`app_rx_valid` 中间不断，`app_rx_last` 与末字节同拍。VL 序号、端口和来源网络在整条消息期间保持不变。接收口不带反压，应用须每拍接收（图 4-3(b)）。

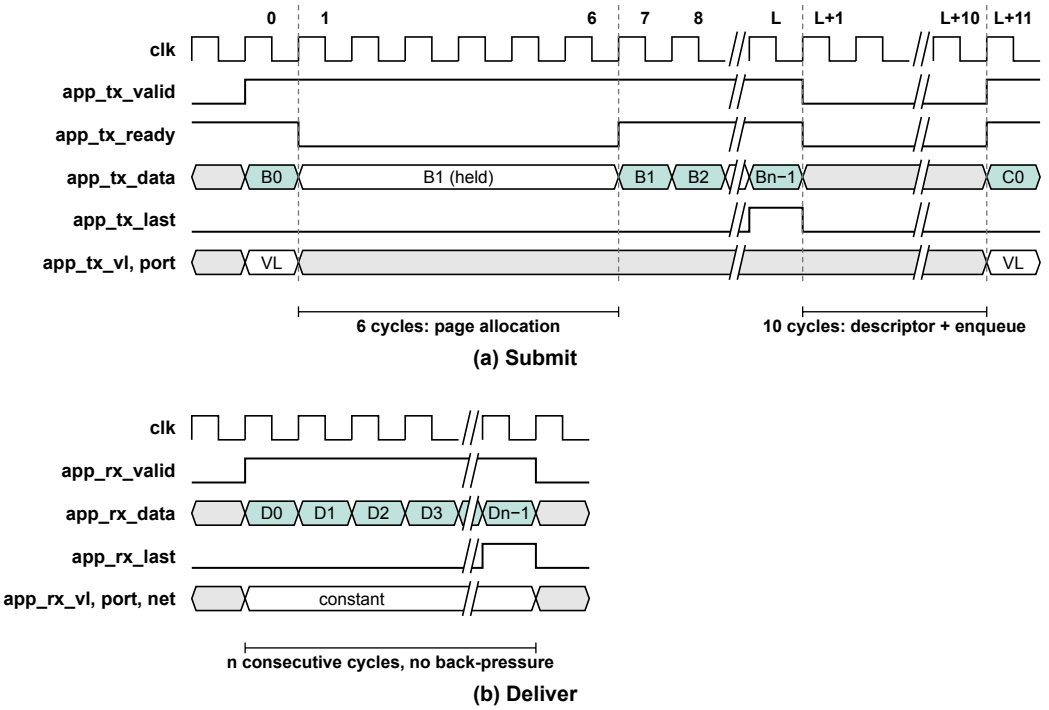



图 4-3 应用口时序，绿色为完成传输的字节（`valid` 与 `ready` 同时有效）：(a) 无服务报文争用时提交一条不跨页的 $n$ 字节消息（$2 \le n \le 512$），首字节采样 VL 与端口，分页 6 拍，末字节后写描述并入队 10 拍；`ready` 为 0 期间应用保持当前字节，C0 为下一条消息首字节，单字节或末字节跨页时末尾多约 6 拍；(b) 交付一条消息，$n$ 个字节连续输出，不带反压

## 4.5 配置总线

配置总线用于写寄存器和三张配置表、读寄存器和计数器，时序如下（$t$ 为周期号，图 4-4）：

- 读：$t$ 拍 `cfg_rd` 为 1，则 $t+2$ 拍 `cfg_rvalid` 为 1、`cfg_rdata` 有效。读不受 `cfg_ready` 限制，复位结束后可以每拍连续读。同一拍读写同一寄存器读到旧值。
- 写寄存器：`cfg_wr` 与 `cfg_ready` 同拍为 1 即完成。

- 写配置表：$t+1$ 拍起 `cfg_ready` 变 0，待写寄存器保持地址和数据，直到核心在某拍 $u$ 给出 `cfg_ok` 时写入，$u+1$ 拍 `cfg_ready` 恢复；不受阻时 $u=t+1$，最长等待 64 拍（第 8.4 节）。

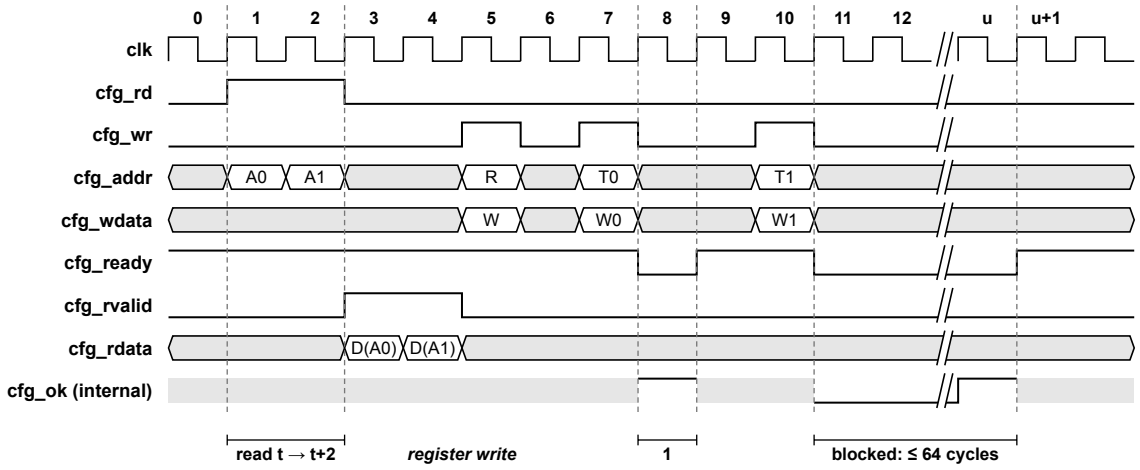



图 4-4 配置总线时序：A 为连续寄存器读（$t$ 拍请求、$t+2$ 拍数据），R 为寄存器写（`cfg_ready` 保持为 1），T 为表写，分别示出不受阻（`cfg_ready` 低 1 拍）和受阻（低至核心给出 `cfg_ok` 的拍 $u$）两种情况。`cfg_ok` 为核心内部信号，灰色表示与本次写无关

## 4.6 事件接口

`ev_*` 端口在每个统计事件发生时给出单周期脉冲，与第 8 章的计数器一一对应，供测试平台逐事件核对。每条发送消息恰好产生 accept 或四种丢弃之一；线上每一帧在其所在网络恰好产生一个类别脉冲，完好帧再恰好产生一个结果脉冲（第 6.6 节）。

## 4.7 内部接口

一级模块之间和核心内部子模块之间的接口以寄存器输出为主；组合输出包括 ready 握手、`cfg_ok`，以及 `afdx_svc` 的数据选择与接收分发直通，后者只做选择和门控。接口按下面四种约定之一工作：

- 字节流握手（`valid/ready/last`）：两者同拍为 1 传送一个字节；附带字段在首字节取样并保持到末字节。
- 请求应答（`req/ack`）：请求保持到应答脉冲，应答与结果同拍给出。
- 单槽交接（`valid/take` 或 `valid/ready`）：生产者置 `valid` 并保持字段不变，消费者取走后才能放入下一项。
- 避让许可（`cfg_ok`）：资源主人用组合信号声明“本拍之后一拍我不访问该表”，写方只在许可拍落下写。

表 4-3 汇总各内部接口，逐信号定义见附录 C。

表 4-3 内部接口汇总

| 接口 | 两端 | 约定 | 关键时序 |
|---|---|---|---|
| 消息 m_* | afdx_svc → tx_buf | 字节流 | 每取一页 m_ready 低 6 拍，消息末尾低约 10 拍，连续为低不超过 96 拍 |
| 入队 enq_* | tx_buf → tx_engine | 请求应答 | 应答不晚于请求后 64 拍 |
| 授权 grant_* | tx_engine → tx_builder | 单槽交接 | 当前帧 tx_sof 那拍起可接下一帧 |
| 起始回报 | tx_builder → tx_engine | 单拍脉冲 | 首个前导字节那一拍的 now |
| 帧字节 s_* | tx_builder ↔ mac_tx | 字节流 | MAC 决定取走时刻，前导末拍取第 0 字节 |
| 帧描述 desc_* | rx_parse → rx_engine | 单槽交接 | f_end 后第 3 拍给出；判决至多 13 拍 |
| 交付队列 rx_dq | rx_engine → rx_deliver | 32 项队列 | 队列非空后第 6 拍首个 d_valid |
| 交付 d_* | rx_deliver → afdx_svc | 字节流，无反压 | 服务在首字节同拍认领 |
| 服务发送 svc_tx_* | 三个服务 → afdx_svc | 字节流 | 按整条消息轮询 |
| 配置表写 | afdx_cfg → 收发核心 | 避让许可 | 等待不超过 64 拍（第 8.4 节） |

# 第 5 章　发送通路

本章按数据流向说明发送核心 afdx_tx_core 的五个部分：分页缓存 afdx_tx_buf 与空闲页池 afdx_tx_free、调度引擎 afdx_tx_engine、组帧器 afdx_tx_builder 和双网 MAC afdx_mac_tx。

## 5.1　发送核心与配置表

图 5-1 是发送核心的框图。应用和服务的消息经 afdx_tx_buf 分页写入载荷 RAM 并登记入队；afdx_tx_engine 按 BAG 选出下一帧并授权给 afdx_tx_builder；组帧器预读载荷、拼接帧头，逐字节交给 afdx_mac_tx 发往两网。

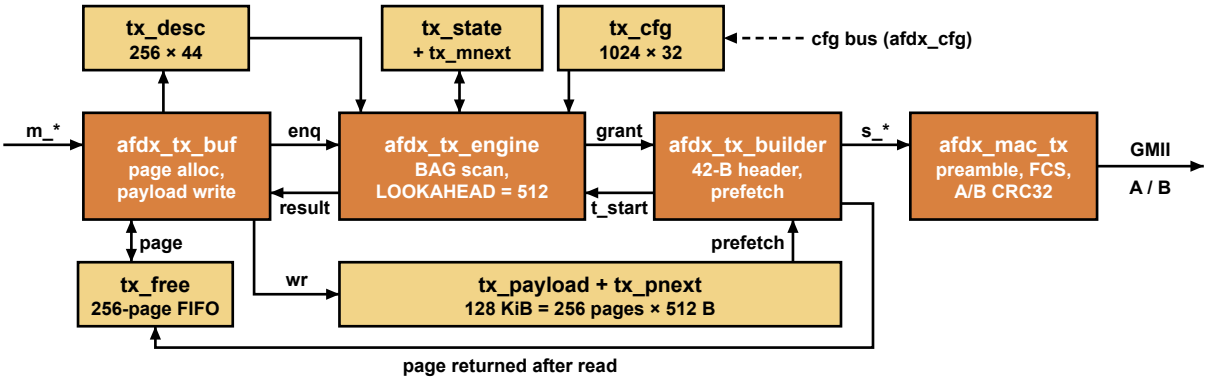



图 5-1　发送核心框图。浅黄色为块 RAM 表；上排为调度引擎读写的表，下排为载荷缓存与空闲页池

发送侧共 7 张表，全部由 afdx_ram_sdp（一写一读、同步读）例化，每张表的写口和读口各只归一个模块，见表 5-1。

tx_cfg 中每条 VL 占 8 个字，字段见表 8-2。六个配置字合成一张 1024×32 的表，比拆成六张浅表少用块 RAM；引擎授权一帧时逐拍读完六个字，时间充裕。

核心外壳。afdx_tx_core 例化 7 张表和 5 个子模块，并持有 tx_cfg 的写口：复位后先用 1024 拍把 tx_cfg 清零，此后只在 cfg_we 与引擎的 cfg_ok 同拍为 1 时把写寄存一拍落表。tx_cfg 清零、空闲页装满和 tx_state 清零三者都完成后 init_done 置 1，引擎和分页缓存才开始工作。

## 5.2　空闲页池

功能。afdx_tx_free 管理载荷缓存的 256 个页号，保证任一页同时只属于一条消息。它是一块 256×8 的 RAM 加读写指针和计数组成的 FIFO，复位后自行把页号 0–255 依次写入，共 257 拍，装完之前不出页。

端口。一个取页口和两个还页口，均带握手：

表 5-1 发送侧配置与状态表

| 表 | 深度 × 位宽 | 写口 | 读口 | 内容 |
|---|---|---|---|---|
| tx_cfg | 1024×32 | 配置总线 | tx_engine | 每条 VL 8 个字，地址 {VL[6:0], 字号 [2:0]} |
| tx_state | 128×63 | tx_engine | tx_engine | timer_active、next_time、sn、q_count、q_head、q_tail |
| tx_mnext | 256×8 | tx_engine | tx_engine | 同一 VL 队列中的下一条消息（首页号） |
| tx_desc | 256×44 | tx_buf | tx_engine | 消息描述：长度、port_en、源/目的端口 |
| tx_pnext | 256×8 | tx_buf | tx_builder | 消息内的下一页 |
| tx_payload | 131072×8 | tx_buf | tx_builder | 128 KiB 载荷，地址 {页号 [7:0], 偏移 [8:0]} |
| tx_free | 256×8 | tx_free | tx_free | 空闲页号 FIFO |

- 取页口 pop_req/pop_ready/pop_ack/pop_ok/pop_page 接 tx_buf，请求应答式；
- 还页口 1 ret1_∗ 接 tx_builder，读完一页还一页；
- 还页口 2 ret2_∗ 接 tx_buf，放弃消息时退页。

状态机。取页由四步状态机完成，见表 5-2。从请求被接受到 pop_ack 共 4 拍，其中 2 拍用于同步 RAM 读出和寄存。

表 5-2 空闲页池取页状态

| 状态 | 动作 | 转移 |
|---|---|---|
| POP_IDLE | pop_ready= 1；接受请求时若 count= 0 立即应答 pop_ok= 0，否则发出读 rd_ptr | 有页 → POP_READ |
| POP_READ | 等待同步 RAM 读出 | → POP_CAPTURE |
| POP_CAPTURE | 寄存读出的页号 | → POP_REPLY |
| POP_REPLY | 给出 pop_ack、pop_ok= 1 和页号 | → POP_IDLE |

还页。每个还页口先把页号收进自己的一个保持寄存器（ret∗_ready 为该寄存器空），再经共用的写口逐个写入 FIFO 尾部，两个寄存器都满时口 1 优先。取页与还页同拍发生时 count 不变。

裕量。FIFO 深度等于页数，任一页号在池中至多出现一次，写入永不溢出；每拍可写入一个页号，两个还页口各自最快每 2 拍还一页，写口不会成为瓶颈。

## 5.3 分页缓存

功能。afdx_tx_buf 从消息接口逐字节收下消息，按页写入 tx_payload，在 tx_pnext 中把同一消息的页串成链，最后写 tx_desc 并向引擎入队。载荷缓存共 256 页、每页 512 字节，地址为 {页号 [7:0], 偏移 [8:0]}。

端口。消息接口 m_*（表 C-1）；取页口和还页口 2；tx_payload、tx_pnext、tx_desc 的写口；入队请求应答口（表 C-2）；五个统计脉冲 ev_accept、ev_drop_dis、ev_drop_full、ev_drop_long、ev_drop_nopage。

状态机。图 5-2 是其状态机，各状态的动作见表 5-3。m_ready 由状态组合产生，只在 IDLE、RECV、DRAIN 三个状态为 1。

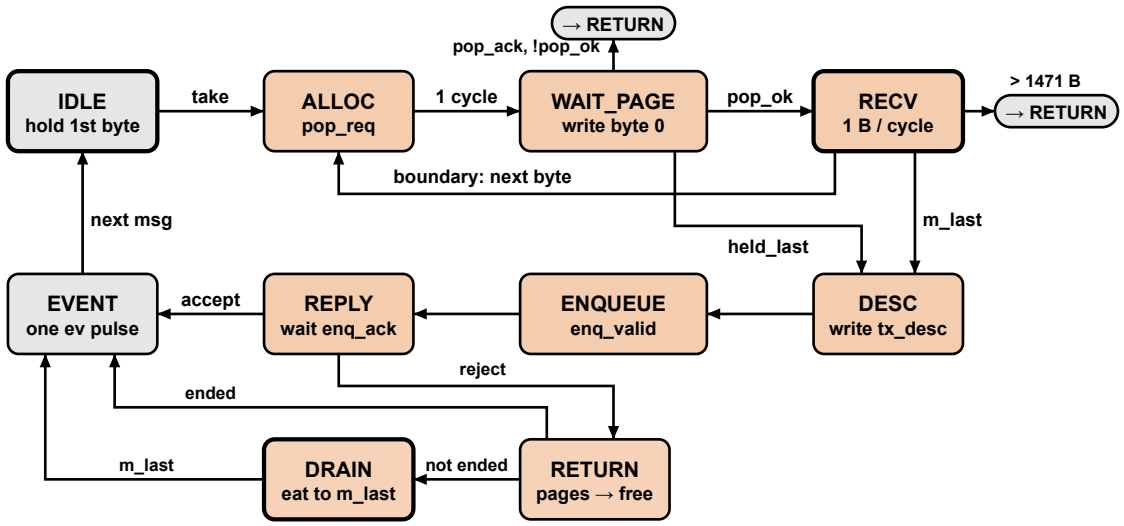



图 5-2 分页缓存状态机。粗框在 init_done 后 m_ready 为 1，字节转移以 m_valid 与 m_ready 同时有效为条件；RETURN、DRAIN 为放弃消息的异常路径，无页与超长转移用标有目标状态的短箭头表示，RETURN 归还全部已用页后退出；省略等待自环

跨页。写满一页的下一个字节不直接写，而是先收进 held_data，取到新页后再写到新页的偏移 0，因此跨页不丢字节，也不需要提前取页。最长消息占 3 页，6 个页寄存器为更小的页长参数留有余量。

关键时序。以首字节被收下为第 0 拍：第 1 拍发取页请求，第 2 拍被空闲页池接受，第 6 拍 pop_ack 到达并写入首字节，第 7 拍回到 RECV。每取一页 m_ready 为 0 共 6 拍。消息末字节之后，DESC、ENQUEUE 各 1 拍，引擎入队 6 拍后应答，EVENT 1 拍，m_ready 共为 0 约 10 拍。

异常处理。超长（第 1472 个字节到来）、取页失败和入队被拒三种情况都走 RETURN：先退回本消息已用的全部页，再收完剩余字节。被丢弃消息占用的页在其统计脉冲之前全部退回，不影响下一条消息取页。reason 只记第一次出现的原因，保证每条消息恰好产生一个统计脉冲。

裕量。每条消息的统计脉冲出现在末字节被收下后 96 拍以内、下一条消息末字节之前，因此第 $k$ 个脉冲对应第 $k$ 条消息。m_ready 连续为 0 的上界由消息末尾决定：$2+64+9+1=76$ 拍，取设计约束 96 拍（第 3.7 节）。一条 1471 字节消息在应用口共占 1499 拍，短于它在线路上的 1538 拍，应用口不是吞吐瓶颈。

表 5-3 分页缓存各状态

| 状态 | 动作 | 转移 |
|---|---|---|
| IDLE | 收下首字节存入 held_data，取样 VL 与端口，长度记 1 | → ALLOC |
| ALLOC | 置 pop_req | → WAIT_PAGE |
| WAIT_PAGE | 等 pop_ack；得页后记入 6 个页寄存器之一，首页记为 enq_head，否则写 tx_pnext[上一页] = 新页；把暂存字节写到新页偏移 0 | 无页 → RETURN；暂存字节是末字节 → DESC；否则 → RECV |
| RECV | 每拍收一字节写入当前页，偏移加 1；偏移到 511 时置 boundary | 已满 1471 字节仍有数据 → RETURN；跨页字节暂存 → ALLOC；末字节 → DESC |
| DESC | 写 tx_desc[enq_head] = {长度, port_en, 端口} | → ENQUEUE |
| ENQUEUE | 置 enq_valid | → REPLY |
| REPLY | 等 enq_ack，记下 enq_result | 收下 → EVENT；拒绝 → RETURN |
| RETURN | 经还页口 2 逐页退回已用页 | 消息已结束 → EVENT；否则 → DRAIN |
| DRAIN | 继续收字节但不保存 | 末字节 → EVENT |
| EVENT | 按原因给出一个统计脉冲 | → IDLE |

## 5.4 调度引擎

功能。afdx_tx_engine 是 tx_state 和 tx_mnext 的唯一主人，负责入队判决、按 BAG 选帧、授权组帧器和根据实际起始时刻更新下一次允许时刻。

状态表。tx_state 每条 VL 一项 63 位，见表 5-4。每条 VL 的消息队列是一条单向链表：q_head、q_tail 存首尾消息的首页号，tx_mnext[首页号] 指向下一条消息，因此 128 条 VL 共用 256 项链表，不需要每 VL 一个 FIFO。

表 5-4 tx_state 字段

| 位 | 字段 | 含义 |
|---|---|---|
| 62 | timer_active | next_time 有效；为 0 表示可立即发送 |
| 61:30 | next_time | 下一帧最早起始的 now 值 |
| 29:22 | sn | 下一帧的序号，复位为 0，之后 1–255 循环 |
| 21:16 | q_count | 排队消息数，不超过配置的 qmax（至多 63） |
| 15:8 | q_head | 队首消息首页号 |
| 7:0 | q_tail | 队尾消息首页号 |

状态机。引擎一次只做一种操作，见图 5-3。IDLE 下按“起始确认 > 入队 > 授权 > 扫描”的优先级选择，各操作按 phase 逐拍推进，做完即回 IDLE，见表 5-5。

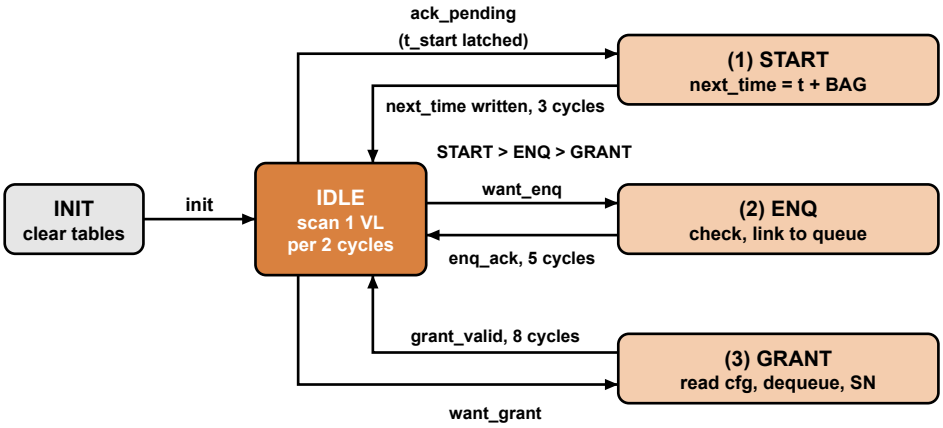



图 5-3 发送调度引擎状态机。使能后按 START、ENQ、GRANT 的优先级分派，三个操作态做完即回到 IDLE；无请求时 IDLE 每 2 拍检查一条 VL。`want_enq` 要求入队请求有效且未处理，并已有候选或扫描配额已满；`want_grant` 要求已有候选、授权端就绪且无待起始帧；省略等待自环与复位转移

表 5-5 调度引擎的操作

| 操作 | 触发 | 逐拍动作 | 拍数 |
|---|---|---|---|
| START | 组帧器报告 $t_{\text{start}}$（先锁存到 `ack_pending`） | 0：读 `tx_state[pend_vl]`；1：保存；2：写 `timer_active`$=1$、`next_time`$=t_{\text{start}}+$`bag`，清 `pend_valid` | 3 |
| ENQ | `enq_valid`，且已有候选或扫描配额已满 | 0：读 `tx_cfg` 字 0 和 `tx_state`；1：保存；2：$d=$`now`$-$`next_time`；3：$d+512$；4：判决，收下则写回 `q_count`$+1$、新队尾，非空队列另写 `tx_mnext`[旧队尾]，并在条件满足时直接定为候选 | 5 |
| GRANT | 有候选且 `grant_ready` | 0–4：依次读配置字 1–5；1：保存字 0 和 `tx_state`；2：读 `tx_desc`、`tx_mnext`[队首]；4：保存描述；7：给出授权，写回 SN 前进、`q_count`$-1$、队首前移，记 `pend_vl` | 8 |
| 扫描 | 无上述请求且无候选 | 每 2 拍发出一条 VL 的读，经 5 级流水判定（图 5-4） | 2/VL |

入队判决。按顺序检查：VL 未启用或两网都未启用回 1；$n$ 大于 `max_payload` 回 3；`q_count` 不小于 `qmax` 回 2；否则收下回 0。收下时若当前没有候选、该 VL 不是 `pend_vl` 且已在 512 拍窗口内，直接定为候选，省去一次扫描。

扫描流水。扫描每 2 拍发出一条 VL 的 `tx_state` 读，结果经图 5-4 的 5 级流水判定。记 $d=$`now`$-$`next_time`，判据为：

- `q_count`$\neq 0$、（`timer_active`$=0$ 或 $d+$`LOOKAHEAD`$\geq 0$）且不是 `pend_vl`：定为候选，扫描停在下一条；
- `q_count`$=0$、`timer_active`$=1$ 且 $d \geq 0$：写回 `timer_active`$=0$，防止 `now` 回绕约 34 s 后把旧时刻误判为将来；
- 判定完成时 `scan_commit` 前进；任一操作插入时流水作废，从 `scan_commit` 重新发出，正在流水中的 VL 不会因操作改写状态而被误判。

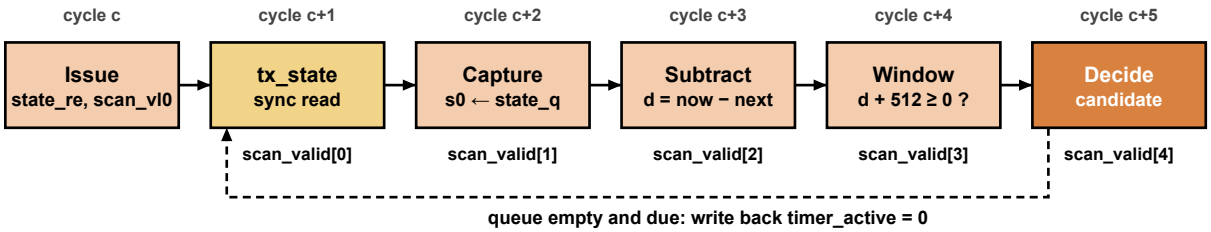



图 5-4 扫描流水。每 2 拍发出一条 VL，第 $c+5$ 拍给出判定；START、ENQ 或 GRANT 插入时清空流水，从 `scan_commit` 重新扫描

公平性。扫描从上次选中的下一条继续，128 条 VL 轮流被检查，不会饿死。入队在已有候选或扫描满 16 条（`SCAN_QUOTA`）时才插入，所以连续入队也不能挡住扫描；无候选时任一 VL 相邻两次被检查的间隔约 400 拍（第 3.7 节）。

**`pend_vl`。**一帧从授权到真正起始之间，该 VL 的 `next_time` 仍是旧值，扫描和入队都不再选它，否则同一 VL 会连发两帧而违反 BAG。组帧器只有一个“下一帧”位置，一个寄存器就够；起始确认写回新的 `next_time` 时清除。

提前选、到点发。128 条 VL 扫描一遍 256 拍，长于最短帧的 84 拍时隙；若到期后才开始找，发送时刻会随扫描位置浮动。引擎在到期前 `LOOKAHEAD`$=512$ 拍内选出候选并完成授权，组帧器备好长度和校验和，`now` 到达允许时刻 $T$ 时 MAC 恰好发出第一个前导字节。回访约 400 拍加授权和准备 22 拍小于 512，见第 3.7 节和图 3-3。

**BAG 的计量。**`next_time` 以实际起始时刻 $t_{\text{start}}$ 加 `bag` 计算，而不是以上一次的允许时刻计算。线路争用使某帧晚发时，下一帧的允许时刻随之顺延，同一 VL 相邻两帧的起始间隔始终不小于 BAG。

**SN。**授权时把当前 `sn` 交给组帧器并写回下一值：复位后每条 VL 第一帧为 0，其后 1、2、…、255、1。

## 5.5 组帧器

功能。`afdx_tx_builder` 把授权的字段组成完整帧内容（目的 MAC 到 SN），在允许时刻交给 MAC，并把实际起始时刻回报给引擎。它内部最多同时有两帧：一帧在 `stream_active` 状态向 MAC 送字节，另一帧在 `pending` 位置准备并等待。`grant_ready`$=$`¬pending`，`pending` 在本帧的 `tx_sof` 那拍清除，因此下一帧的授权可在当前帧起始后立即进入，准备在当前帧发送期间完成。

数据通路。组帧器的结构见图 5-5：帧头走 336 位移位寄存器，载荷走预读流水和 8 项字节 FIFO，补零和 SN 由计数器产生，三者经一个选择器送出。

准备。收下授权的同拍锁存全部字段和 `USER_ID`，IP 标识取 16 位计数器并加 1。之后 10 拍算 IP 头校验和，每拍只做一次加法，见表 5-6。

起始。起始条件为 `prepared`、上一帧已送完，且 `grant_timed`$=0$、允许时刻已过或距允许时刻至多 4 拍。4 拍提前量补偿起始路径上的四级寄存器：`now` 寄存 1 拍、差值 `now_q`$-T$ 寄存 1 拍、`s_valid` 寄存 1 拍、MAC 判断起始并寄存输出 1 拍。于是线路

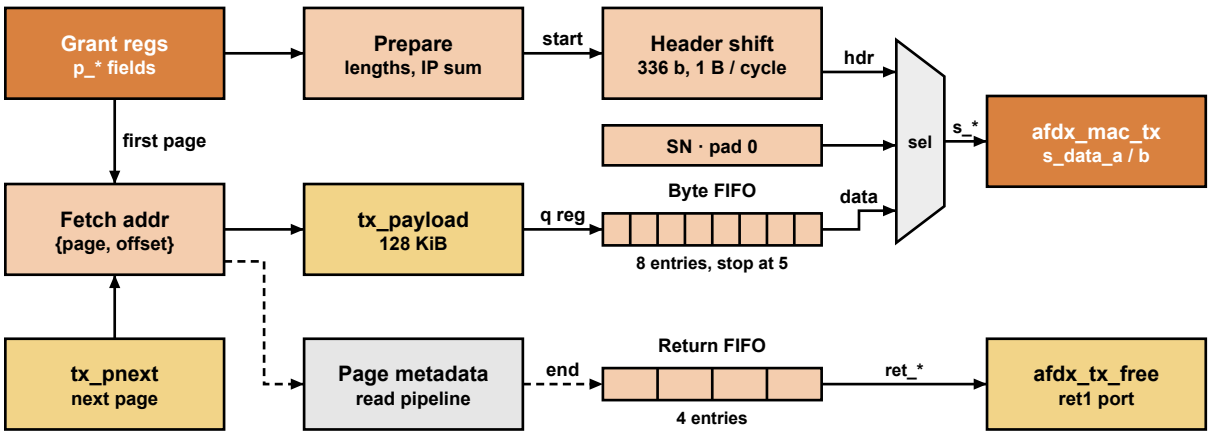



图 5-5 组帧器数据通路。帧头与载荷分两路，在选择器汇合后交给 MAC；页归还由独立的读元数据流水触发，与载荷进入字节 FIFO 同时发生，不等待 FIFO 消费完该页

表 5-6 组帧器准备步骤

| prep | 动作 |
|---|---|
| 0 | IP 总长 $= n + 28$，UDP 长 $= n + 8$；累加器置初值 `0x4500+0x0111=0x04611` |
| 1–6 | 依次累加 IP 总长、标识、源 IP 高/低 16 位、目的 IP 高/低 16 位 |
| 7–8 | 两次把高 4 位进位加回低 16 位 |
| 9 | 取反得校验和，置 `prepared` |

空闲时第一个前导字节恰在 `now`$= T$ 那一拍出现。起始拍把 42 字节帧头并行装入移位寄存器，A、B 两路共用，只在第 11 字节（接口号）B 网换为 `40`。

发送。MAC 每取一字节，按 `part` 依次送出帧头 42 字节、载荷 $n$ 字节、$n < 17$ 时补零到 17 字节、带 `s_last` 的 SN。`tx_sof` 那拍的 `now` 作为 $t_{\text{start}}$ 回报引擎。

载荷预读。起始拍起，只要 FIFO 中不足 5 字节就每拍发出一个 `tx_payload` 读；读请求寄存、同步 RAM 和输出寄存共 3 拍后写入 FIFO。当前页与下一页号分开保存：起始时读 `tx_pnext`[首页]，每次跨页再读下一页的后继，跨页时请求连续不断。读到页末字节或消息末字节时，该页号随数据一起进入 4 项归还 FIFO，经还页口 1 退回空闲页池。

异常处理。MAC 报 `tx_underrun` 时本帧记为坏帧，不产生 `ev_tx_a`/`ev_tx_b`；其余帧在 `tx_eof` 时按本帧启用的网络各给一个脉冲。

裕量。

- **FIFO** 不溢出。发出读时 FIFO 中至多 4 字节，另有至多 3 个读在途，最多 $4+3+1 = 8$ 字节，等于 FIFO 深度。
- 不欠载。MAC 在起始后第 8 拍才取第 0 个字节，帧头又占 42 拍，第一个载荷字节在起始后第 50 拍才被取走，而 FIFO 在起始后第 4 拍就开始填充；此后每拍补入一字节、取出一字节。下一页号在跨页前 500 拍以上已读出，跨页不产生空拍。
- 准备时间。准备 11 拍，远小于当前帧至少 84 拍的发送时间，下一帧在当前帧结束前已就绪。

从消息末字节被收下到 tx_sof 的名义值为 31 拍，上界 100 拍，逐拍分解见第 3.7 节图 3-3。

## 5.6 双网发送 MAC

afdx_mac_tx 用一个状态机驱动 A、B 两套 CRC 和输出寄存器，两网天然同拍。它是发送节拍的主人：上游只负责备好字节，何时取走由 MAC 决定。图 5-6(a) 是其状态机，(b) 是第 6.2 节的接收 MAC。

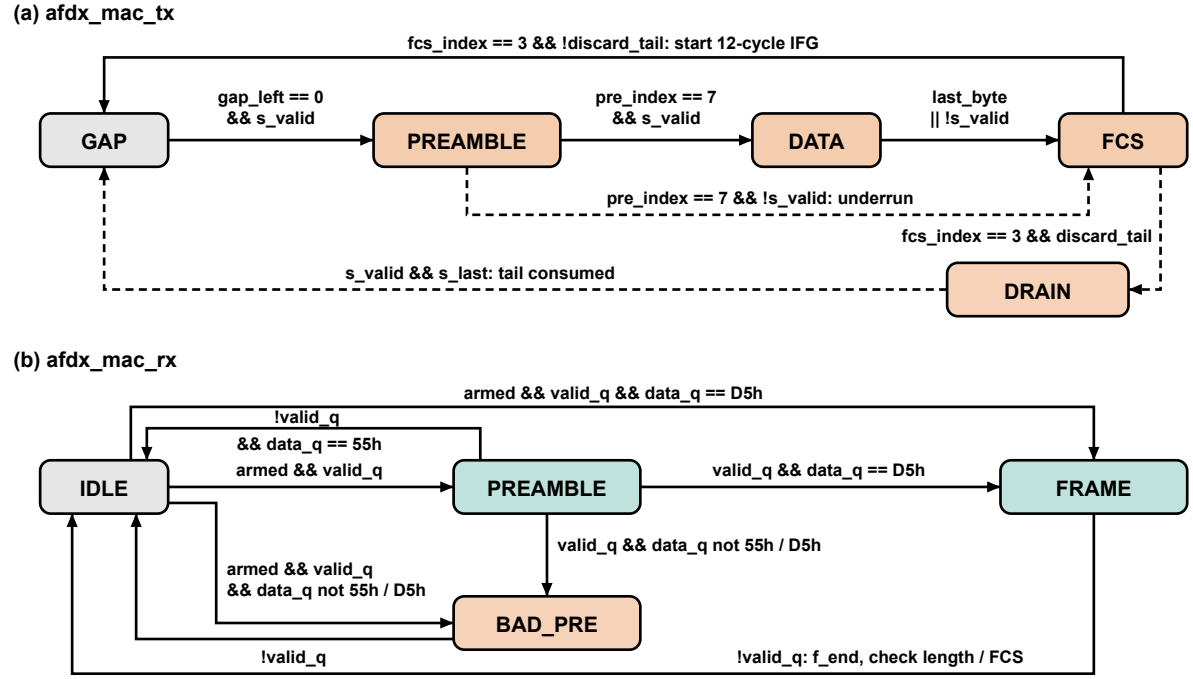



图 5-6 双网 MAC 状态机：(a) 发送，仅在 byte_en 有效时转移，虚线为欠载保底路径；(b) 接收，条件使用寄存后的有效标记和数据，PREAMBLE 或 BAD_PRE 因有效标记撤销而退出时报告前导错误。省略逐字节工作自环、等待自环与复位转移

各状态的动作见表 5-7，发送一帧的字节时间见表 5-8，$L$ 为取走的字节数。起始那一拍取样 s_en_a、s_en_b；未启用的那一网整帧期间 tx_valid 和 tx_data 保持 0，两网都未启用时照常走完节拍，使各帧时序与网络选择无关。上游一直有帧等候时，帧间隔恰为 12 拍。

数据阶段若上游未备好字节（欠载），MAC 给出 tx_underrun 脉冲，发出 4 字节不取反的错误 FCS，使对端必然校验失败，然后吞掉上游这一帧的剩余字节。按设计欠载不会发生，这是保底措施。复位期间线路输出全为 0，复位结束后先等满 12 拍帧间隔再允许首帧。

选择自研 MAC 的依据在于结构：它不经用户侧 FIFO，空闲时上游备好帧即发出前导码，接收侧逐字节剥离 FCS，A、B 两网共用一个状态机而天然同拍。eLinx 生成的三速以太网 IP 带用户侧 FIFO、保留 FCS，且每网一个实例。资源采用 2026-10-11 同器件单口装箱对照：自研 MAC 556 个 LUT、198 个触发器、不占存储块，该份 IP 配置为 1400 个 LUT、975 个触发器、24 个存储块。

表 5-7 发送 MAC 各状态

| 状态 | 动作 | 转移 |
|---|---|---|
| GAP | 输出 0；`gap_left` 递减 | `gap_left`$=0$ 且 `s_valid` $\rightarrow$ PREAMBLE，同拍取样两网使能、给 `tx_sof` |
| PREAMBLE | 输出 7 个 `55` 和 `D5`；末拍 `s_ready`$=1$ 取走第 0 字节 | 取到字节 $\rightarrow$ DATA；未备好 $\rightarrow$ FCS（欠载） |
| DATA | 每拍输出一字节并更新两网 CRC | 末字节已出 $\rightarrow$ FCS；未备好 $\rightarrow$ FCS（欠载） |
| FCS | 输出 4 字节 FCS，低字节先发；第 4 拍后撤 `valid`、给 `tx_eof`，`gap_left`$=11$ | 正常 $\rightarrow$ GAP；欠载 $\rightarrow$ DRAIN |
| DRAIN | `s_ready`$=1$，吞掉上游剩余字节 | 收到 `s_last` $\rightarrow$ GAP |

表 5-8 MAC 发送的字节时间

| 字节时间 | 线上内容 | 本拍动作 |
|---|---|---|
| 起始前一拍 | 空闲 | 判断起始，取样两网使能 |
| 0–6 | `55` | |
| 7 | `D5` | 取走第 0 个字节 |
| 8 至 $8+L-1$ | 第 0 至 $L-1$ 个字节 | 取走下一字节，取走带 `s_last` 的字节后停止 |
| 其后 4 拍 | FCS | A、B 各算各的 |
| 再后至少 12 拍 | 空闲 | 第 12 拍可判断下一帧起始 |

# 第 6 章　接收通路

接收核心 afdx_rx_core 每网一套 MAC 和解析器，两网共用一个接收引擎和一个交付模块。帧在 MAC 中逐字节校验 CRC，在解析器中边收边检查帧头并把载荷试写进本网的环形缓冲；完好帧的描述交给引擎做 VL 查表、完整性检查和冗余管理，引擎的决定使试写的数据提交或回退；提交的消息由交付模块逐字节送给应用。整条通路每拍处理一个字节，不缓存整帧。

## 6.1　接收核心与配置表

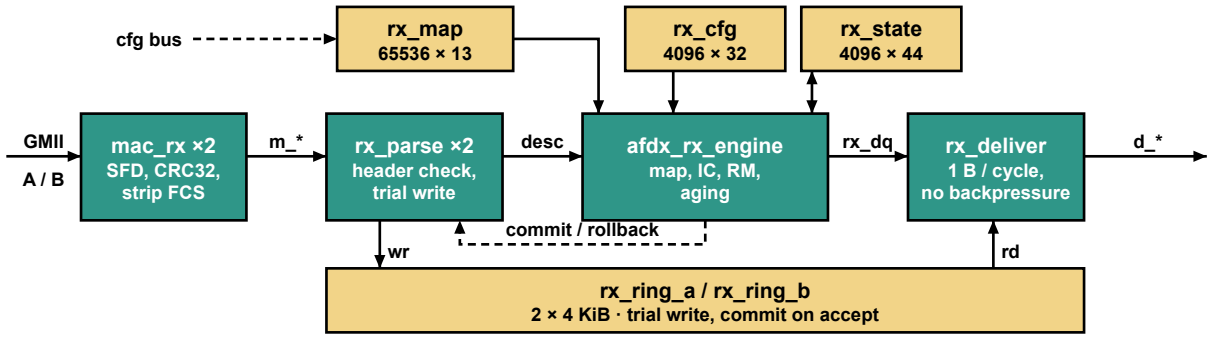



图 6-1　接收核心框图。浅黄色为块 RAM 表；虚线为引擎对试写数据的提交或回退

接收侧的表见表 6-1。rx_map 以 16 位 VLID 直接寻址，任意 4096 个 VLID 都能映射到接收 VL 序号 idx；读出后直接进一级寄存器，查表固定 2 拍。按综合后估算，写成一个 65536×13 的数组与拆成 16 块占用的存储资源相同，前者少用约 200 个逻辑单元，因此不拆分。

表 6-1　接收侧配置与状态表

| 表 | 深度 × 位宽 | 写口 | 读口 | 内容 |
|---|---|---|---|---|
| rx_map | 65536×13 | 配置总线 | rx_engine | valid[12]、idx[11:0] |
| rx_cfg | 4096×32 | 配置总线 | rx_engine | en、net_a、net_b、ic_en、rm_en、skew_max[16:5]、max_payload[27:17] |
| rx_state | 4096×44 | rx_engine | rx_engine | 每网 PSN 及有效位；rm_last、rm_time 及有效位 |
| rx_ring_a/b | 4096×8 | 本网 rx_parse | rx_deliver | 每网一个环形缓冲，按到达顺序存载荷 |
| rx_dq | 32 项寄存器 | rx_engine | rx_deliver | 待交付的网、起点、长度、idx、UDP 端口 |

skew_max 的单位是 now_c 的一个刻度（32.768 µs）。

核心外壳。`afdx_rx_core` 例化两套 MAC 与解析器、一个引擎、一个交付模块和 5 张表，并持有 `rx_map`、`rx_cfg` 的写口。复位后用 65536 拍同时清 `rx_map`（每拍一项）和 `rx_cfg`（前 4096 拍），引擎另清 `rx_state`，全部完成后 `init_done` 置 1。之后表写只在 `cfg_ok` 为 1 时寄存一拍落表。各模块的统计脉冲在外壳中相或后寄存一拍输出。`rx_dq` 的队头选择先在四组各做 8 选 1，寄存后再 4 选 1，满足第 9 章“多于 8 选 1 分两拍”的规则。

## 6.2 接收 MAC

`afdx_mac_rx` 把 GMII 输入寄存一拍后，在 `rx_valid` 为 1 的每一拍处理一个字节，状态机见图 5-6(b)。

- 找帧起始符。收到 `55` 继续等待，收到 `D5` 后下一字节即帧首字节；`D5` 前的 `55` 可以是任意个。收到其他值，或未见 `D5` 时 `rx_valid` 就回 0，判前导错。复位后若 `rx_valid` 已为 1，先等它回 0，不从半帧开始收。
- 剥离 **FCS**。每收一字节都累加 CRC 和帧长；输出比输入晚 4 字节，收到第 $k+4$ 字节时输出第 $k$ 字节，最后 4 字节（FCS）不送后级。
- 帧结束。寄存后的 `rx_valid` 首次为 0 即结束，给 `f_end` 和 `f_err`：依次判前导错、帧长不在 64–1518、CRC 余式不等于 `0xDEBB20E3`，否则完好。超过 1514 字节后不再输出，按帧长错结束。
- 帧间。`f_end` 之后立即回到空闲，两帧之间只有 1 个低电平周期也能正确分开。

关键时序。GMII 上最后一个 FCS 字节记为第 0 拍：第 1 拍它进入输入寄存器，第 2 拍寄存后的 `rx_valid` 为 0，第 3 拍给出 `f_end`、`f_err` 和 `f_len`。去掉 FCS 的最后一个内容字节在第 2 拍从 `m_data` 输出，比第 3 拍的 `f_end` 早 1 拍，解析器据此完成校验和累加。

## 6.3 帧头解析与试写缓存

`afdx_rx_parse` 按字节位置边收边检查，不保存整个帧头，结构见图 6-2：一个字节位置计数器 `position` 驱动各项检查，两个小状态机分别负责长度计算和帧尾定类。

检查项。下列任一条不满足即为“帧头错”：

- 第 0–3 字节为 `03 00 00 00`；第 12–13 字节为 `08 00`；第 14 字节为 `45`；第 23 字节为 `11`；
- IP 不分片（MF 为 0、片偏移为 0）；IP 头 10 个 16 位字的反码和为 `FFFF`；
- 记 $n$ 为 UDP 长度减 8，要求 $1 \le n \le 1471$，IP 总长等于 $n+28$，且含 FCS 的帧长等于 $47+\max(n,17)$；
- UDP 校验和非 0 时，伪头、UDP 头与 $n$ 字节数据的反码和为 `FFFF`。

补零字节的值不检查。解析器同时记录 VLID、源/目的端口、$n$ 和 SN。

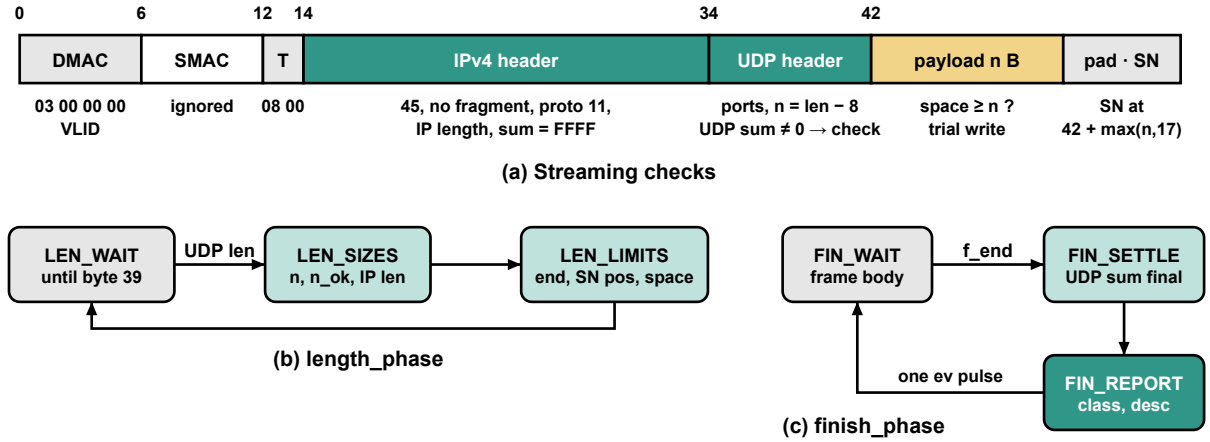



图 6-2 帧头解析：(a) 按字节位置检查并试写载荷，不缓存整帧或帧头；(b) 收到 UDP 长度后的两拍长度计算；(c) 帧尾的两拍定类

长度计算。第 39 字节收到 UDP 长度低字节后，`length_phase` 用两拍算出 $n$、长度是否合法、期望 IP 总长（LEN_SIZES），再算载荷末位置、SN 位置、期望帧长和环形缓冲所需空间（LEN_LIMITS），第 42 字节到来时全部就绪。

校验和。IP 头的 10 个 16 位字和 UDP 伪头、UDP 头、载荷分别用 20 位和 26 位累加器每两字节加一次；进位折叠用两级寄存器流水，与累加并行，帧尾后第 1 拍（FIN_SETTLE）UDP 校验和最终稳定。

先试写、后提交。第 42 字节起的载荷边收边写进本网环形缓冲，但不立即生效。环形缓冲有三个指针（图 6-3）：读指针 `rd` 归交付模块；已提交写指针 `wr_commit` 和试写指针 `wr_trial` 归本网解析器。剩余空间按 `wr_commit` 计算，正在接收的帧不计入。第 42 字节到来时 $n$ 已知，若剩余空间小于 $n$ 则本帧不写，记为溢出。

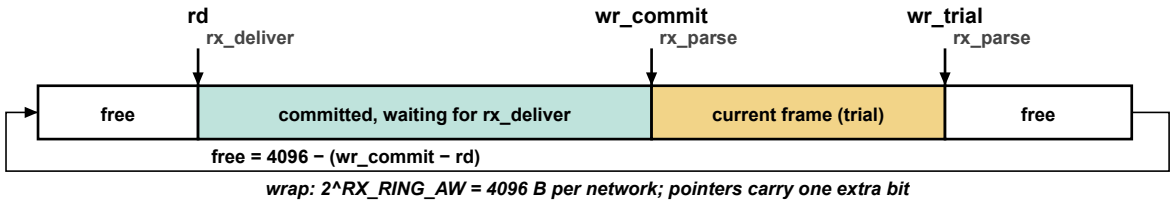



图 6-3 接收环形缓冲的三个指针。通过全部检查的帧使 `wr_commit` 前进到 `wr_trial`；否则 `wr_trial` 退回 `wr_commit`

定类别。每个 `f_end` 定一次类别：`finish_phase` 在 `f_end` 后第 2 拍进入 FIN_REPORT 作出判定，类别脉冲和帧描述都寄存输出，在第 3 拍有效。判定顺序为：`f_err` 为 FCS 错或帧长错（含前导错）时直接定类；否则依次看帧头错、溢出、完好。类别不是完好时，解析器自行把试写指针退回本帧开始前，不交给引擎，所以线上来多密的坏帧都不会漏计，引擎也只需处理完好帧。完好帧的帧描述见表 C-4；两网在同一拍结束的完好帧，描述也在同一拍给出。

裕量。同一网相邻两帧的 `f_end` 至少相隔 73 拍（最短帧 64 字节加前导 8 字节和 1 拍间隔），而引擎处理一份描述 13 拍、两网各一份也只要 26 拍，帧描述单槽在下一份到来前必被取走。

第一个前导字节出现在清表完成之前的帧整帧忽略，不给描述也不计数。

## 6.4 完整性检查与冗余管理

`afdx_rx_engine` 是 `rx_state` 的唯一主人，一次只处理一份帧描述或做一步老化，两网描述同拍到达时先处理 A 网。逻辑上的处理流程见图 6-4：查 `rx_map`，读 `rx_cfg` 和 `rx_state`，依次做配置检查、完整性检查、冗余管理和交付登记，每一步失败都走向对应的丢弃计数并回退试写。

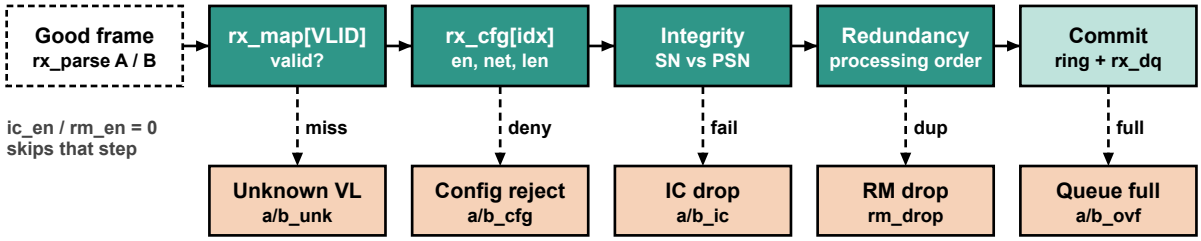



图 6-4 接收引擎对一份帧描述的判决流程。浅橙色为丢弃，各回退试写并产生一个对应统计位脉冲

状态机。引擎有 INIT、IDLE、FRAME、AGE 四个操作态，FRAME 按 `phase` 0–11 逐拍推进，每拍至多一级比较，见图 6-5 和表 6-2。不通过的帧提前跳到写回或报告步，因此一份描述的处理时间为 5–13 拍（含 IDLE 取描述的 1 拍）。

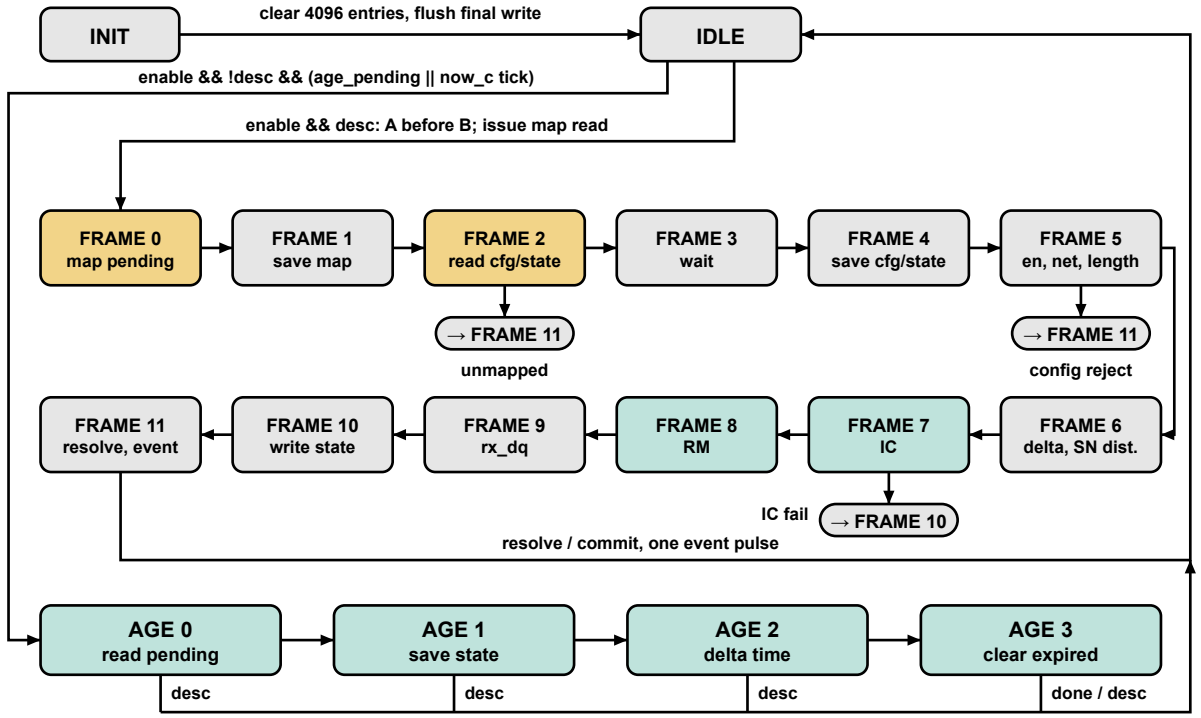



图 6-5 接收引擎状态机。FRAME 的 12 个 phase 沿箭头逐拍推进，标有目标 phase 的短箭头表示提前结束路径；AGE 仅在无描述符时逐项检查冗余记录，每个 phase 均可因新描述符到达而返回 IDLE，`desc` 表示 `desc_a_valid` 或 `desc_b_valid`；省略等待自环与复位转移

配置检查。`rx_map` 无效计“未知 VL”；VL 未启用、本网未启用或 $n$ 大于 `max_payload` 计“配置不允许”。

表 6-2 接收引擎 FRAME 各步

| phase | 动作 |
|---|---|
| （IDLE） | 取走帧描述（take），以 VLID 读 rx_map |
| 0–1 | 等待同步读出，保存映射 |
| 2 | 映射无效 → 11（未知 VL）；否则以 idx 读 rx_cfg 和 rx_state |
| 3–4 | 等待读出，保存配置和状态 |
| 5 | VL 或本网未启用、$n >$max_payload → 11（配置不允许）；同时算 PSN 的下一个、到达时间差、SN 与 rm_last 的原始距离 |
| 6 | 时间差取绝对值，算 PSN 的下下个，原始距离折算到 1–255 环上 |
| 7 | 完整性检查，更新本网 PSN；不通过 → 10 |
| 8 | 冗余管理判定 deliver_allowed |
| 9 | 判重复；rx_dq 满记溢出；否则登记交付队列，更新 rm_last、rm_time |
| 10 | 状态有变化则写回 rx_state |
| 11 | 给出 resolve 与 commit，按结果给一个统计脉冲，回 IDLE |

完整性检查（ic_en= 1 时）每网独立进行。记本网上一帧的序号为 PSN，下列任一条成立即通过：清表后本网首次收到该 VL；SN 为 0（对端复位）；SN 是 PSN 的下一个；SN 是 PSN 的下下个（允许丢一帧）。“下一个”在 1–255 上循环，0 的下一个是 1。无论通过与否，都把本网 PSN 更新为本次 SN；不通过计“完整性检查丢弃”。

冗余管理（rm_en= 1 时）按处理顺序交付首个有效副本；两网描述同拍待处理时先处理 A 网。副本到达间隔在 skew_max 内时，同一消息只交付一次，判据如下：

- 尚无记录，或到达时刻与 rm_time 之差超过 skew_max：交付；两份逆序处理时 16 位差取绝对值。
- 否则 SN 等于 rm_last：另一网的重复帧，丢弃并计“冗余管理丢弃”。
- 否则 SN 为 0 或 rm_last 为 0：交付。
- 否则在 1–255 环上，SN 领先 rm_last 1–127 步则交付，其余丢弃。

交付时若 rx_dq 已满则计“溢出”并回退；否则提交环形缓冲、登记 rx_dq，并更新 rm_last 和 rm_time。一份描述的处理至多 13 拍；两网描述同拍到达时，后处理的一份从描述有效到作出决定不超过 27 拍，设计约束取 64 拍。

判决顺序。每份副本独立地先做完整性检查，通过后再做冗余管理，符合 ARINC 664 P7 的处理顺序；一网丢帧不影响另一网的序号判断。完整性检查不通过时 PSN 照样更新，使该网在下一帧恢复同步。

老化。now_c 每变化一次，引擎在空闲时用 AGE 的 4 步检查下一条 VL；有帧描述到来时立即放下，回 IDLE 先处理帧，之后重做这一项。判据为：若记录已超过 0x4000 个刻度（约 0.537 s），清除 rm_time_valid。帧描述优先，被打断的同一项会重试，4096 项约 0.134 s 轮完一遍，记录在约 0.671 s 内失效，早于 16 位时间差的半圈 1.074 s，因此计时回绕不会把旧记录误判为窗口内。

## 6.5 交付

`afdx_rx_deliver` 按先后从 `rx_dq` 取一项，从对应环形缓冲每拍读出一字节送出，读过的空间随即释放，结构见图 6-6。

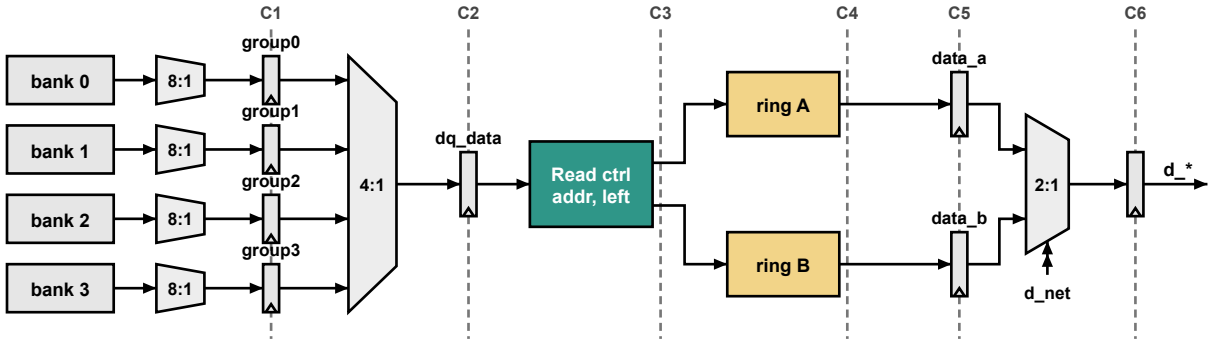



图 6-6 交付模块。32 项寄存器队列分 4 组，先组内 8 选 1、寄存后再 4 选 1。灰色虚线 C1–C6 依次标识组内选择寄存、队头寄存、读请求寄存、同步 RAM 读出、两网数据寄存和输出寄存六个采样边界；前两拍对应 `HEAD_IDLE` 与 `HEAD_SELECT`，`HEAD_RELEASE` 是等待请求撤销的控制态，不增加数据流水级

取队头。队头选择分两拍（HEAD_IDLE 组内 8 选 1，HEAD_SELECT 组间 4 选 1），同时给出 `dq_pop`；队头进入 `dq_data` 后，读控制在下一拍发出第一个读，之后每拍地址加 1，`left` 减到 0 停止。读请求、同步 RAM、数据寄存三级流水由 `valid_pipe`、`last_pipe` 对齐，第一个 `d_valid` 在 `dq_count` 变为非零后第 6 拍出现。`d_last` 那一拍重新允许取队头。

一条消息的 $n$ 个字节在连续 $n$ 拍内送出，`d_last` 与末字节同拍；VL 序号、端口和来源网络在整条消息期间不变。交付口不带反压。线路空闲时帧尾到首个 `d_valid` 名义 24 拍、设计上界 200 拍（图 3-3）；前一消息未完时，取队头与读流水共约 6 拍，紧跟其末字节之后 16 拍内开始。32 项描述队列用于接住长载荷交付期间到达的短帧（第 3.7 节）。

## 6.6 统计分类

线上每一帧（一段连续的 `rx_valid` 为 1）在其所在网络恰好得到一个类别脉冲，由解析器在帧尾离开线路后 32 拍内给出；类别为完好的帧再恰好得到一个结果脉冲，由引擎在决定后 8 拍内给出，且不晚于帧尾后 150 拍。每网的 `ev` 位定义见表 6-3。

位 7 由解析器（环形缓冲放不下）和引擎（`rx_dq` 已满）两处相或。同一网上解析器给出第 $k$ 帧的脉冲时，引擎对第 $k-1$ 帧的判决早已结束（相邻帧尾至少相隔 73 拍，判决至多 27 拍），两处不会同拍，相或不丢计数。两网合计另有 `ev_rm_drop`（冗余管理丢弃）和 `ev_deliver`（交付）。这些脉冲与第 8 章的计数器一一对应，测试平台据此逐帧核对每一帧的去向。

表 6-3　接收统计脉冲 ev_a、ev_b

| 位 | 名称 | 给出者 | 含义 |
|---|---|---|---|
| 0 | 完好帧 | rx_parse | 通过 MAC 和帧头检查 |
| 1 | FCS 错 | rx_parse | CRC 不符 |
| 2 | 帧长错 | rx_parse | 帧长越界、超长或前导错 |
| 3 | 帧头错 | rx_parse | 固定字段、IP/UDP 长度或校验和错 |
| 4 | 未知 VL | rx_engine | rx_map 无效 |
| 5 | 配置不允许 | rx_engine | VL 或本网未启用，或超过 max_payload |
| 6 | 完整性检查丢弃 | rx_engine | SN 不连续 |
| 7 | 溢出 | 两处相或 | 环形缓冲放不下，或 rx_dq 已满 |

# 第 7 章　服务功能

RTC、SNMP 和 615A 三项服务都以 UDP 报文工作，不另设 MAC 或专用 VL 通道，而是经服务通道 `afdx_svc` 与应用共享同一对收发核心。服务报文因此与用户报文遵守同一套 BAG 调度、完整性检查和冗余管理。

## 7.1　服务通道

`afdx_svc` 由组合选择和五个状态寄存器组成：发送侧为 `busy`、`owner`、`turn`，接收侧为 `rx_active`、`rx_claimed`，结构见图 7-1。

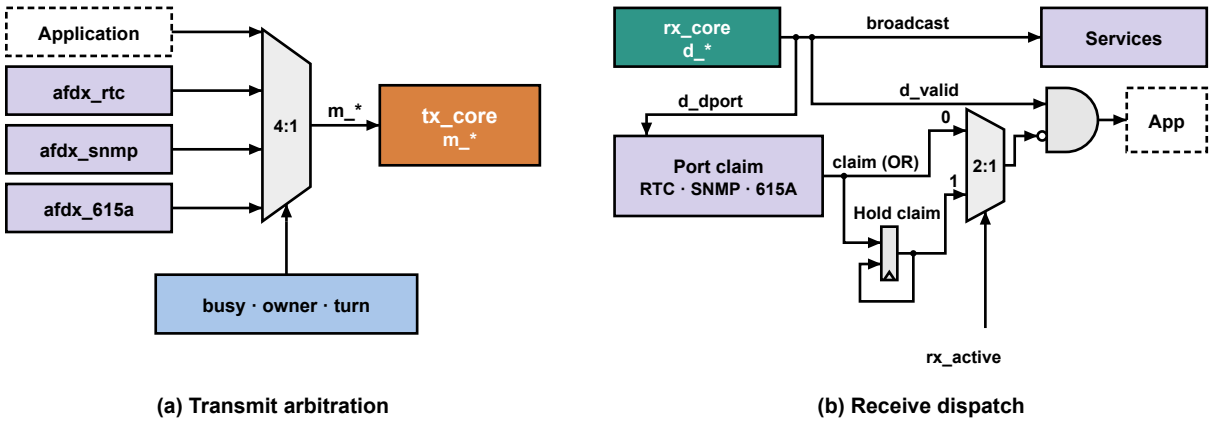



图 7-1　服务通道：(a) 应用与三个服务按整条消息轮询共享发送核心；(b) 交付消息广播给服务，首字节有效时保存认领结果，其余周期保持。`rx_active` 选择当前认领或寄存的认领结果，反相后与 `d_valid` 相与形成应用口有效信号；图中省略直通应用口的数据与元数据连线

发送仲裁。应用口与三个服务发送口按整条消息轮询，次序为应用、RTC、SNMP、615A。选中一条消息后保持到末字节握手，消息中途的停顿不允许其他来源插入；完成后从下一来源开始选择。有服务报文等待时，应用不能连续独占；没有服务发送时，应用的数据和 ready 组合直通发送核心，不增加缓存和流水拍。仲裁状态只有三个寄存器：`busy` 表示一条消息正在传送，`owner` 记其来源，`turn` 记下次从哪个来源开始查找；末字节握手时 `busy` 清零、`turn` 指向当前来源的下一个。由此任一来源在提出请求后，至多等其余三个来源各传完一条消息。

接收分发。接收核心的交付口同时广播给应用和三个服务。各服务按 UDP 目的端口给出认领信号，首字节的认领结果保持到末字节：被认领的消息只送服务，应用口的 valid 和 last 为 0；其余消息原样送给应用。三个服务使用不同的 UDP 端口，且都不对交付口施加反压。认领信号由各服务按 `d_dport` 组合产生，首字节那一拍直接决定应用口的 `valid`，之后由寄存的 `rx_claimed` 保持，所以分发不增加交付延迟。

## 7.2 RTC 时钟同步

`afdx_rtc` 维护 32 位秒加 32 位纳秒的实时时钟，每拍前进 8 ns，纳秒满 $10^9$ 向秒进位；秒的起点为 2000-01-01 00:00:00 UTC，线上 NTP 秒为其加 3155673600。同步采用 RFC 4330 的 SNTPv4 主从方式，交互见图 7-2。

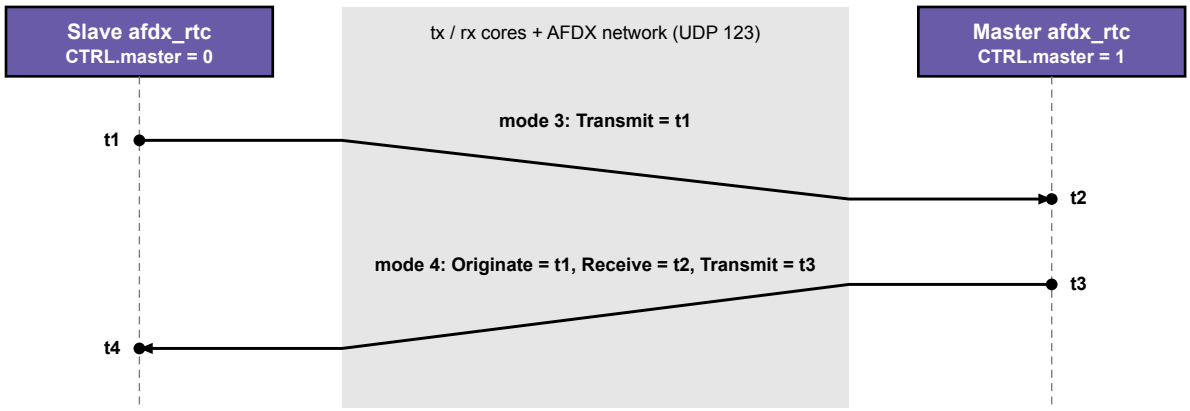



图 7-2 SNTP 主从对时交互。中间灰带合并收发核心与 AFDX 网络；RTC 轴上的实心点为采样时刻，$t_1$、$t_3$ 在消息首字节发送握手时采样，$t_2$、$t_4$ 在核心交付首字节时采样。回应的 Originate 字段逐位复制请求的 Transmit 时间戳

结构。`afdx_rtc` 包含本地时钟、一个 48 字节接收移位寄存器、发送字节状态机、校时计算状态机和三个 `afdx_ntp_fraction` 换算器（发送时间戳编码、接收或参考时间戳编码、回应时间戳解码），两个状态机见图 7-3。

从端每隔 `PERIOD` 发一个请求，记下发送时刻 $t_1$；主端在收到请求时记 $t_2$，回应发送时记 $t_3$，并把请求的 Transmit 时间戳逐位复制到回应的 Originate 字段；从端收到回应时记 $t_4$。从端只接受 Originate 与本次请求逐位相等的回应，随后计算

$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \tag{7-1}$$

并加到当前时钟。一个周期内未收到有效回应时，下一周期以新的 Transmit 时间戳重发，直至校时成功。

时间戳采样。发送时间戳在消息首字节与发送核心握手时采样，接收时间戳在核心交付首字节时采样；时间戳在握手后开始换算，并在对应字段发出前完成，不预测排队时间。

发送时序。发送状态机每字节用 4 拍：TX_GROUPS 先在两组字段中各选一个 32 位字，TX_WORD 选字，TX_BYTE 选字节并置 `tx_valid`，HANDSHAKE 等 `tx_ready`。一个 48 字节报文至少 192 拍。Transmit 字段位于第 40–47 字节，距首字节握手至少 160 拍，而换算只需 65 拍，所以时间戳必在字段发出前就绪。主端的 Receive 字段 $t_2$ 在请求首字节到达时开始换算，回应要等整条请求收完才开始发送，同样有充足余量。

无乘除法器的换算。NTP 时间戳的小数部分是 32 位二进制小数，而本地时钟以纳秒计。`afdx_ntp_fraction` 用 32 轮、每轮 2 拍的逐位比较减法计算 $\lfloor \text{ns} \times 2^{32} / 10^9 \rfloor$，反方向用逐位加与右移并在末轮四舍五入。每拍最多一级 32 位加减，整纳秒时间戳往返换算

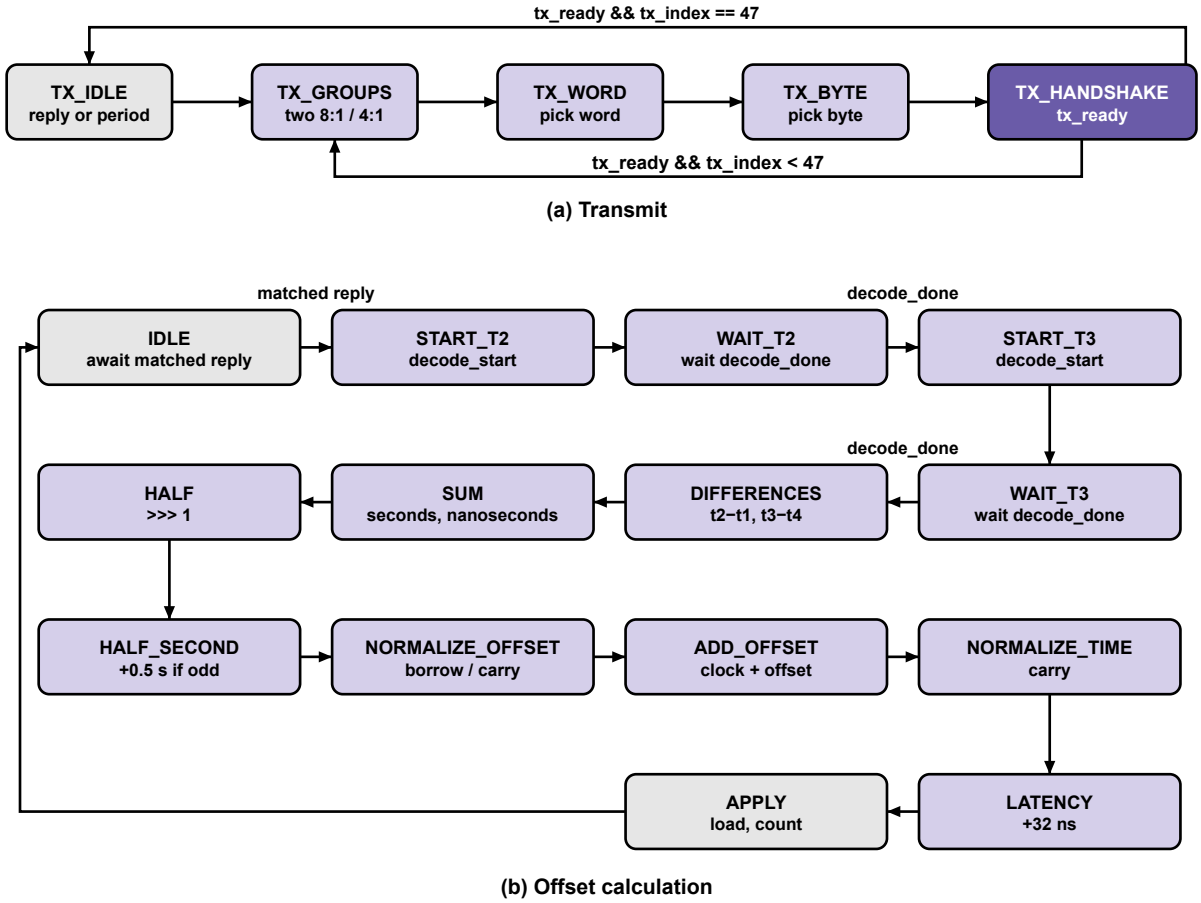



图 7-3 RTC 的两个状态机：(a) 报文发送，无反压时每字节 4 拍，主端有待回应请求、从端倒计时归零时启动；(b) 校时计算，由匹配的回应触发，每拍至多一级算术运算。状态名省略 `CALC_` 前缀；省略等待自环与复位转移

得到原值。偏差计算也把秒差和纳秒差分开，各拍至多一级 32 位运算，步骤见图 7-3(b)：先依次解码回应中的 $t_2$、$t_3$ 小数部分（各 65 拍），再用 9 拍算出两段差、求和、除以 2，并把奇数秒的半秒折入纳秒、处理借位进位。校正量在 ADD 步加到当前时钟，到 APPLY 步写回共经过 4 拍，所以 LATENCY 步补回 32 ns；一次校时约 140 拍，期间时钟照常前进。

回应的接受条件。从端只在下列条件同时成立时启动计算：版本为 4、模式为 4、本端正在等待回应且发送状态机空闲（重发中的请求尚未发完时不接受），Originate 与本次请求的 Transmit 逐位相等。迟到的旧回应因 Originate 不符而被丢弃，不会用旧的 $t_1$ 计算。计算期间或 `rx_done` 有效的周期不发起新请求。

报文。请求与回应均为 48 字节，所有多字节字段高字节先发，见表 7-1。双方 UDP 源、目的端口均为 `UDP_PORT`（默认 123）。

寄存器。RTC 寄存器位于 `0x80`–`0x88`，见附录表 B-1。写时间先写 `SEC`、`NS` 两个暂存字，再以 `CTRL` 的装入位一次生效；读 `SEC` 时同时快照纳秒，随后读 `NS` 得到同一时刻的值。偏差以 `OFFSET_SEC`$\times 10^9+$`OFFSET_NS` 纳秒表示。

表 7-1 SNTPv4 报文字段

| 字节 | 请求（从端） | 回应（主端） |
|---|---|---|
| 0 | `0x23`：LI=0、VN=4、mode=3 | `0x24`：LI=0、VN=4、mode=4 |
| 1 | stratum=0 | stratum=1 |
| 2–3 | Poll=4、Precision=−26 | 同左（约 14.9 ns） |
| 4–11 | Root Delay、Root Dispersion 均为 0 | 同左 |
| 12–15 | 0 | Reference ID=`LOCL` |
| 16–23 | 0 | 最近一次配置装入的时刻 |
| 24–31 | Originate=0 | 请求 Transmit 的逐位复制 |
| 32–39 | Receive=0 | $t_2$ |
| 40–47 | Transmit=$t_1$ | $t_3$ |

## 7.3 SNMP 网络管理

`afdx_snmp` 是服务通道上的 SNMP 代理，认领 UDP 目的端口 161，回应的目的端口为请求的源端口，经配置的发送 VL 送出。图 7-4 是其处理流程。

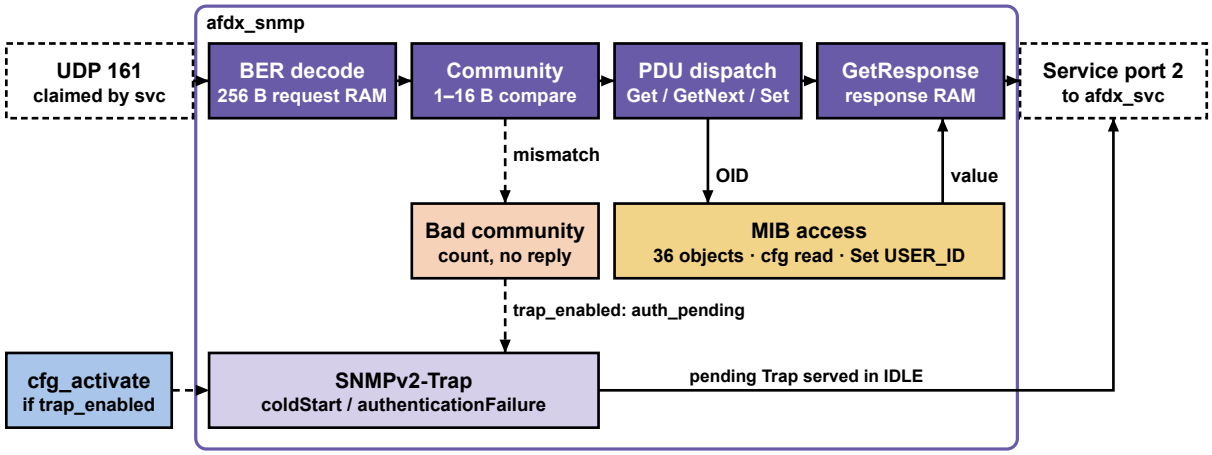



图 7-4 SNMP 代理处理流程

状态机。代理的主状态机有 44 个状态，按功能分为 7 组，见图 7-5。请求先完整收进请求 RAM，之后的解析和查找都通过“取字节”子程序读 RAM：调用方把返回状态写入 `return_state`，子程序用 4 拍完成越界检查、发读、等同步读出和保存，再跳回调用方。这样每个解析状态只处理一个已就绪的字节，单拍逻辑只有一级比较。

**BER** 逐字节编解码。请求先整条写入 256×8 的请求 RAM，再逐字节解析 SEQUENCE、version、community 和 PDU；PDU 内依次是 request-id、error-status、error-index 和变量绑定表。长度字段支持短形式和 81、82 两种定长形式。回应在 256×8 的回应 RAM 中逐字节组装，原样保留 request-id 和 community，再经服务发送口输出。

支持的操作。

- 版本：SNMPv1（version=0）和 v2c（version=1）。

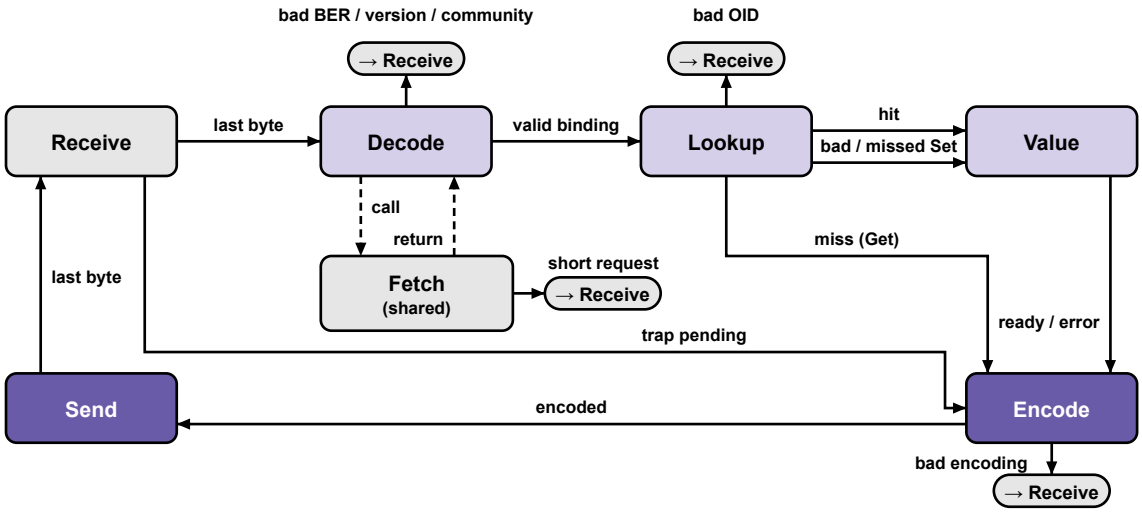



图 7-5 SNMP 代理状态机（按功能分组）。虚线为共享取字节子程序 Fetch 的调用与返回，该子程序也供 Lookup、Value 和 Encode 使用；异常返回用标有 Receive 的短箭头表示；Set 未命中经 Value 组的 `SET_ERROR` 编码错误应答；省略组内转移、等待自环与复位、禁用转移

- **Get / GetNext**：Get 精确匹配标量 OID；GetNext 按子标识符逐个比较，返回严格大于请求 OID 的第一个对象，可从 `1.3` 遍历全部对象。查不到时，v1 回 noSuchName，v2c Get 回 noSuchObject，GetNext 到末尾回 endOfMibView。
- **Set**：唯一可写对象为 `USER_ID`（Gauge32，0–65535）。代理先检查 OID、类型和取值范围，再经配置总线写入并读回，以读回值组装回应。只读对象、类型不符和越界分别按 v1/v2c 的错误码回应，配置不变。
- **Trap**：发送 SNMPv2-Trap（PDU `A7`）到 UDP 端口 162，含 sysUpTime.0 和 snmpTrapOID.0 两个变量绑定。`TRAP_ENABLE` 已置位且 `ENABLE` 从 0 变为 1 时发 coldStart；community 不符且 `TRAP_ENABLE` 已置位时发 authenticationFailure。

每个请求支持一个变量绑定；community 不符时不回应并计数 `BAD_COMMUNITY`。

**OID** 比较。查找按对象表顺序逐个尝试 36 个候选，每个候选按子标识符逐段比较：多字节子标识符按 BER 的 7 位分组累加还原成整数后再比，而不是按编码字节比较，因此 GetNext 的“严格大于”按 OID 的数值顺序成立。私有子树中 424242 这样的多字节子标识符也正确排序。

回应组装。LEN1–LEN4 四拍自内向外算出值、变量绑定、绑定表、PDU 和整条消息的长度，BUILD 再按“类型、长度、内容”逐字段写入回应 RAM；community、OID 和回显的值用 COPY 状态从请求 RAM 逐字节复制。回应写完后由发送状态逐字节读出，每字节 4 拍送入服务发送口。

并发。代理一次处理一个请求，只在 IDLE 接收新请求；处理期间到达的请求仍被认领并计入请求数，但不回应，由管理端按超时重发。超过 256 字节的请求整条丢弃。

管理对象。MIB 共 36 个对象：sysDescr、sysUpTime 两个系统对象，以及私有子树 `1.3.6.1.4.1.424242.3` 下的 34 个对象，覆盖版本号、`USER_ID`、RTC 秒和同步次数、全部 27 个收发统计计数器、SNMP 请求数、community 错误数和 Trap 发送数。对象值不另

存副本，代理经与主机共享的配置读通路读取寄存器：主机访问其他地址时主机优先，同一地址的主机读和代理读在同一拍合并；代理写只在主机无读写且 `cfg_ready` 为 1 时授予。

寄存器。SNMP 寄存器位于 `0x90–0xA3`，包括启用、回应 VL、community 长度和内容、请求计数、Trap 开关与计数及 sysDescr，见附录表 B-1。

## 7.4 615A 数据加卸载

`afdx_615a` 实现 ARINC 615A 的目标端：加载端经 TFTP 送来上传清单，目标主动读取头文件和数据文件，校验后把数据留在 8 KiB 加载区，并通过状态文件报告进度与结果；目标也能以介质定义卸载把加载区作为文件 `MEMORY.BIN` 读出。协议文件版本默认 `A4`，兼容 `A3`；目标标识为 `DEMC3_1`。

### 7.4.1 结构

`afdx_615a` 的主状态机约 70 个状态，按图 7-6 分组：收到的报文先整条存入 1 KiB 的接收缓冲，由 DISPATCH 按操作码、端口（TID）和块号分派给六类处理过程；处理过程读文件内容时调用“读字节/读整数”子程序，发报文时调用“报文输出”子程序，后者在发送 RAM 中逐字节组好报文、送出后跳回 `after_tx` 指定的状态，通常是 WAIT_PACKET。数据和文件检查失败时调用 `fail(reason)`，发送 TFTP ERROR 和失败状态文件；状态文件传输失败（含重试耗尽）直接结束。请求名称或模式错误只回 ERROR；LCI/LNO 在初始化阶段拒绝。

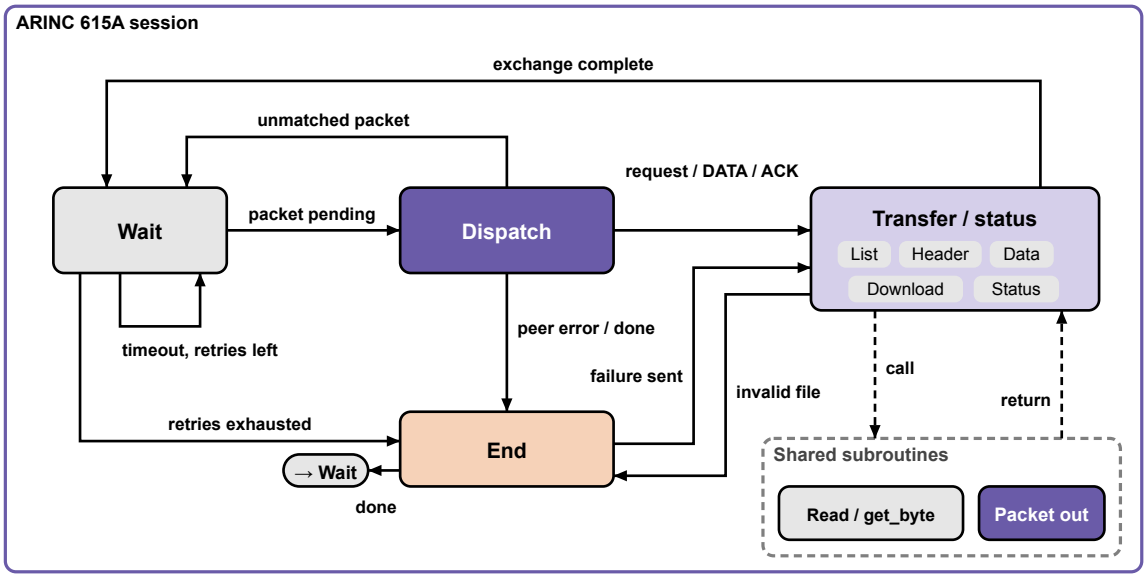



图 7-6 615A 目标状态机（按功能分组）。Transfer/status 合并 List、Header、Data、Download 与 Status 处理；共享读字节和报文输出子程序合置于虚线框，以一对虚线表示调用与返回，两者也供其他组使用；End 完成后用标有 Wait 的短箭头返回；省略组内转移、等待自环与复位转移

接收缓冲只存一个报文：报文未被取走时到达的下一报文仍被认领但不写入。TFTP 停等协议保证对端在收到回应前不发下一报文，偶发的重复报文由对端超时重发恢复。

### 7.4.2 TFTP

TFTP 实现 RRQ、WRQ、DATA、ACK、ERROR 五种报文，模式 octet，每块 512 字节，块号从 1 开始，WRQ 以 ACK0 建立传输。初始请求的目的端口为 `REQUEST_PORT`（默认 59），此后双方使用各自的传输标识符（TID），目标 TID 从 `0x8100` 起递增；主文件传输与状态文件传输各自保存 TID 和块号。报文在 RAM 中保留到回应到达；超时（默认 2 s）后原样重发上一报文，重传上限默认 1 次，耗尽以原因 5 失败。重复 DATA 只重发 ACK，不重复写 RAM。请求中的 RFC 2347 选项被忽略，不返回 OACK。

### 7.4.3 上传

上传流程见图 7-7(a)：

1. 加载端读取 `DEMC3_1.LUI`，目标返回接受状态 0001。
2. 目标写 `DEMC3_1.LUS` 报告初始状态。
3. 加载端写 `DEMC3_1.LUR` 上传清单，含头文件名和件号；版本不支持则以原因 1 拒绝。
4. 目标发 LUS 0002 后读取清单中的 LUH 头文件；头文件校验通过后读取其列出的数据文件。
5. 数据块按序写入加载区并累计 CRC。首个满块写入后暂缓 ACK，先经独立传输发送一次进度 LUS，再确认数据块。
6. 末块（短块或空块）到达后，长度和 CRC 均正确才确认，并发送 LUS 0003、进度 100；否则发送 TFTP ERROR 和 LUS 1003。

文件检查。LUH 支持 ARINC 665-1 的 8002 版本：读取头文件字数、数据文件列表指针、数据文件数、文件名、数据文件字数和 CRC-16，检查头文件实际长度、版本、列表数量、头文件 CRC 及数据文件的长度和 CRC。CRC-16 多项式 `0x1021`，最高位优先、初值 `FFFF`、结果不取反，以 8 位并行的组合函数每拍处理一字节。每份新 LUS 的计数器加 1，重传不加；进度为已写字节占预期总字节的整数百分比，成功前最多 99。百分比不用除法器：先用移位相加把已写字节乘 100，再反复减去预期总字节计商，最后反复减 10 拆成十进制字符。

### 7.4.4 介质定义卸载

卸载流程见图 7-7(b)。加载端读 `DEMC3_1.LND`，目标回应后发初始 LNS；加载端写 LNR 列表，只接受 `MEMORY.BIN`；目标发 LNS 0002 后主动写 `MEMORY.BIN`，按 512 字节块停等发送最近一次成功上传的内容，首个满块确认后报告一次进度；末块确认后发 LNS 0003。卸载只读加载区，不改变已加载字节数。

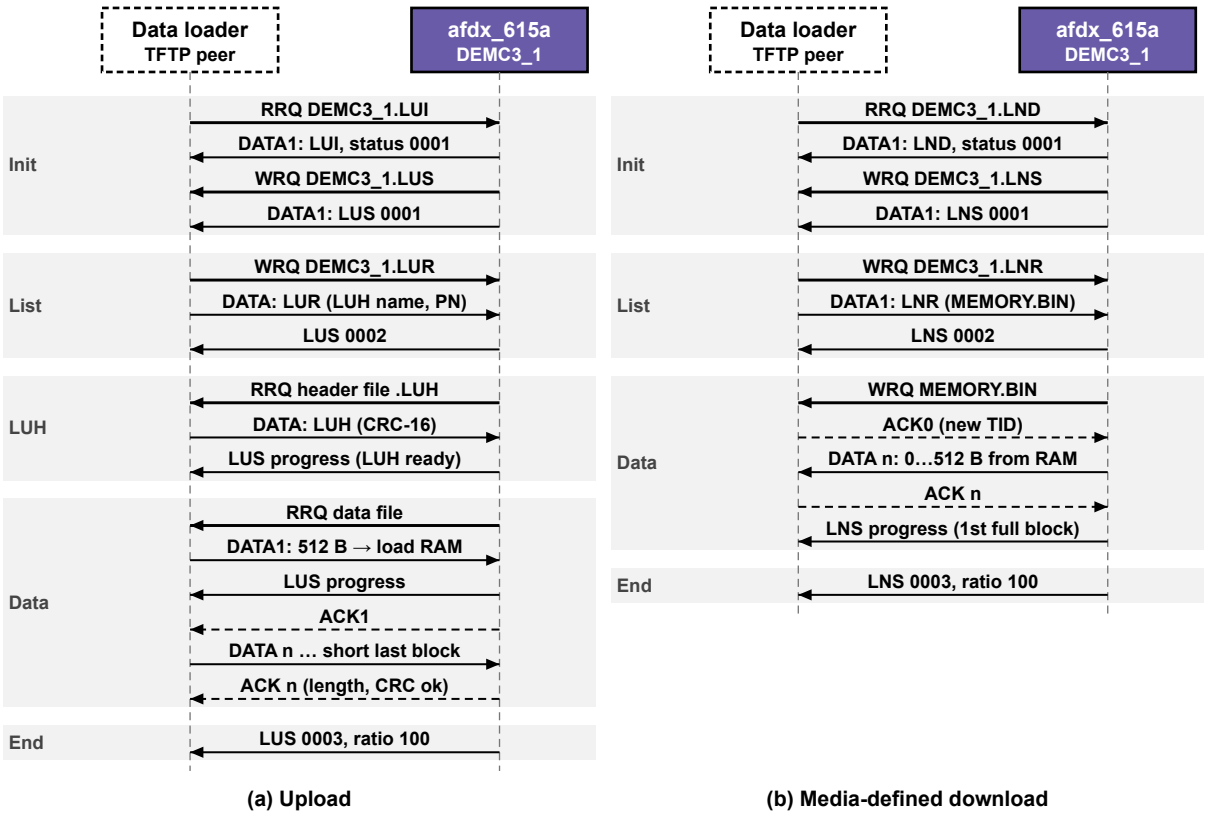



图 7-7 615A 交互：(a) 上传，LUH 校验完成后先报告进度，再请求数据文件；(b) 介质定义卸载。两者末块均少于 512 字节，整倍数长度补零长度块；粗线为 RRQ/WRQ 请求，细线为 DATA 与状态文件，虚线为 ACK；省略初始化、列表、头文件的部分 ACK 及重复状态文件的 WRQ/ACK 握手

上传和卸载互斥，操作在最终状态文件获 ACK 后才记为成功；新上传开始时撤销旧的有效长度。数据仅保存在 RAM 中，不执行程序或写入非易失存储。

### 7.4.5 存储与寄存器

五块 RAM 分别存放接收报文（1 KiB）、发送与重传报文（1 KiB）、LUR（512 B）、LUH（2 KiB）和加载数据（8 KiB），均以 `afdx_ram_sdp` 例化。寄存器位于 `0xB0–0xBB`，见附录表 B-1；`STATUS` 给出空闲、进行中、成功、失败和原因码。原因码 0 为无错误，1 为元数据不支持或无效（版本、文件名、数量），2 为容量或长度超限，3 为 CRC 错，4 为对端 TFTP ERROR 或模式错误，5 为重传耗尽。加载区可经 `MEMORY_ADDRESS`、`MEMORY_DATA` 逐字节读回。

# 第 8 章　配置、复位与统计

`afdx_cfg` 负责配置总线的地址译码、寄存器、27 个统计计数器、读数据选择，以及写配置表时与收发核心的握手。它的输出全部来自寄存器，结构见图 8-1。

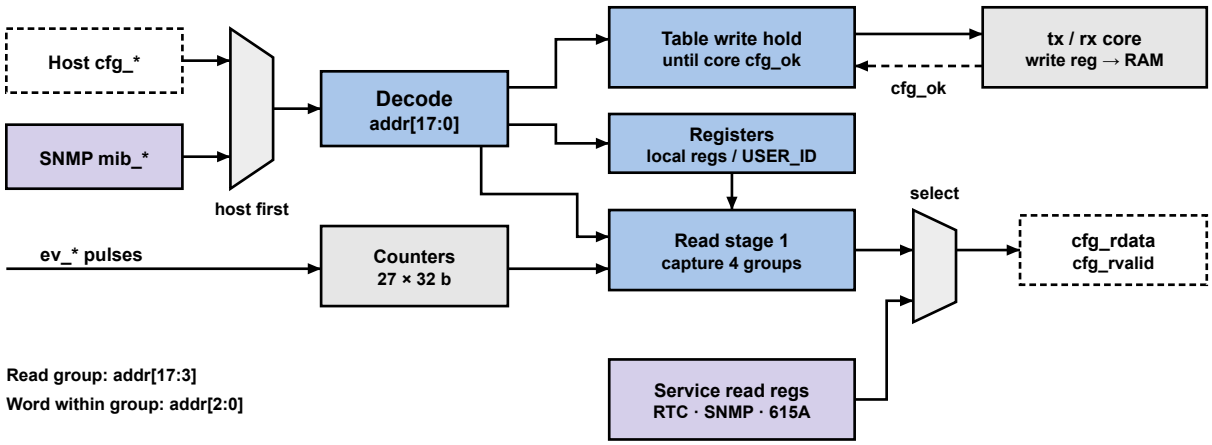



图 8-1　配置模块。主机与 SNMP 代理共享一条总线；表写在保持寄存器中等待核心许可；$t$ 拍读请求在 $t+1$ 拍锁存本地与服务读寄存器，$t+2$ 拍选出数据

## 8.1　地址映射

配置地址为 18 位字地址，映射见表 8-1。没有全局使能，每条 VL 有自己的 `en` 位。表以外的地址没有映射，写无效果，读出 0。

表 8-1　配置地址映射

| 地址 | 读写 | 内容 |
|---|---|---|
| `0x00000` | 只读 | VERSION，`0xC3110000` |
| `0x00001` | 只读 | STATUS，位 0 为 `tx_ready` |
| `0x00002` | 读写 | USER_ID[15:0]，填入源 MAC 第 9–10 字节 |
| `0x00040`–`0x00046` | 只读 | 发送计数器：accept、drop_dis、drop_full、drop_long、drop_nopage、tx_a、tx_b |
| `0x00048`–`0x0004F` | 只读 | A 网接收计数器，对应 `ev_a` 位 0–7 |
| `0x00050`–`0x00057` | 只读 | B 网接收计数器，对应 `ev_b` 位 0–7 |
| `0x00058`、`0x00059` | 只读 | rm_drop、deliver |
| `0x0005A`、`0x0005B` | 只读 | a_gdrop、b_gdrop（保留，恒为 0） |
| `0x00080`–`0x000BB` | — | RTC、SNMP、615A 寄存器（附录表 B-1） |
| `0x01000` + VL×8 + 字号 | 只写 | `tx_cfg` |
| `0x10000` + VLID | 只写 | `rx_map`，数据取位 12:0 |
| `0x20000` + idx | 只写 | `rx_cfg`，idx < 4096 |

三张表的译码只看高位：地址位 17:10 为 `0000_0100` 是 `tx_cfg`，位 17:16 为 `01` 是 `rx_map`，位 17:12 为 `10_0000` 是 `rx_cfg`。

## 8.2　配置表格式

三张配置表的字段见表 8-2。复位后三张表全部清零，即全部 VL 未启用、全部 VLID 未映射。

表 8-2　配置表字段

| 表与字 | 位 | 字段 | 含义 |
|---|---|---|---|
| `tx_cfg` 字 0 | 0 | `en` | VL 启用 |
| | 1、2 | `net_a`、`net_b` | 从 A、B 网发送 |
| | 8:3 | `qmax` | 队列最多排队的消息数，0–63 |
| | 19:9 | `max_payload` | 最大消息长度，1–1471 字节 |
| `tx_cfg` 字 1 | 31:0 | `bag` | BAG，单位为拍，1 ms = 125000，不超过 $2^{30}$ |
| `tx_cfg` 字 2 | 15:0 | VLID | 目的 MAC 低 16 位 |
| `tx_cfg` 字 3、4 | 31:0 | 源 IP、目的 IP | IPv4 地址 |
| `tx_cfg` 字 5 | 31:16、15:0 | 源端口、目的端口 | 消息不带端口时使用的默认 UDP 端口 |
| `tx_cfg` 字 6、7 | — | 保留 | 写 0 |
| `rx_map`[VLID] | 12 | `valid` | 该 VLID 已映射 |
| | 11:0 | `idx` | 接收 VL 序号，0–4095 |
| `rx_cfg`[idx] | 0 | `en` | VL 启用 |
| | 1、2 | `net_a`、`net_b` | 接收 A、B 网的帧 |
| | 3、4 | `ic_en`、`rm_en` | 启用完整性检查、冗余管理 |
| | 16:5 | `skew_max` | 两网最大到达时间差，单位 32.768 µs |
| | 27:17 | `max_payload` | 最大载荷，1–1471 字节 |

## 8.3　读写通路

读。读数据的多路选择分两拍完成，读延迟固定为 2 拍。第 1 拍按地址低 3 位同时从四组源（单个寄存器、发送计数器、A 网计数器、B 网计数器）各选一个值寄存，并按地址位 17:3 记下组号；三个服务也在同一拍把各自的读值寄存。第 2 拍按组号从这些寄存值中选一个送到 `cfg_rdata`，同时给出 `cfg_rvalid`。每拍至多一级 8 选 1，满足第 9 章的选择器规则。

写。`cfg_ready` 为 1 时 `cfg_wr` 在同一拍完成：寄存器和服务寄存器直接写入；表地址则装入保持寄存器并拉低 `cfg_ready`，等对应核心的 `cfg_ok`（第 4.7 节）。`tx_ready` 为 0 时保持寄存器清空、`cfg_ready` 为 0，清表期间的写不被接受。

与 **SNMP** 共享。SNMP 代理读写管理对象使用同一条总线。顶层的仲裁规则是：主机本拍写，或读不同地址时主机优先；主机读与代理读同一地址时合并为一次读，两者各自在 2 拍后取得数据；代理写只在主机无读写且 `cfg_ready` 为 1 时授予。读数据通路由两套 2 拍有效标记分别对齐，主机的读时序不受代理影响。

## 8.4 配置写与查表的避让

配置表的读口归收发引擎，写口归配置总线。若同一拍对同一 RAM 读写同一地址，结果不确定。本设计不做旁路，而是让配置写“避开”引擎的读：每个引擎输出一个组合信号 `cfg_ok`，表示“这一拍我不会发出该表的读”：

$$\text{发送侧：}\ \ \text{cfg_ok} = \text{enable} \wedge (\text{op} = \text{IDLE}) \wedge \neg\text{want_enq} \wedge \neg\text{want_grant} \tag{8-1}$$

$$\text{接收侧：}\ \ \text{cfg_ok} = \text{enable} \wedge (\text{op} \in \{\text{IDLE}, \text{AGE}\}) \wedge \neg\text{desc_a_valid} \wedge \neg\text{desc_b_valid} \tag{8-2}$$

核心外壳中的写寄存器只在 `cfg_ok` 为 1 时收下写。引擎的读使能寄存器和外壳的写使能寄存器在同一时钟沿装入、下一拍作用到 RAM；`cfg_ok` 为 1 的那一拍引擎不装入读使能，所以下一拍 RAM 上只有写。

`cfg_ok` 只用到引擎自身的寄存器和兄弟模块直接来自寄存器的输出（入队请求、帧描述有效、`grant_ready`），组合深度很浅。

`afdx_cfg` 收下一次表写后保持 `tx_cfg_we` 或 `rx_map_we`/`rx_cfg_we` 及地址、数据，直到对应核心的 `cfg_ok` 为 1；核心外壳在该拍把写寄存进自己的写口寄存器，下一拍作用到 RAM，`afdx_cfg` 同时撤销写请求并恢复 `cfg_ready`。时序为：

$$\text{cfg_ready}(t{+}1) = 0, \quad u = \min\{\, \tau \geq t{+}1 : \text{cfg_ok}(\tau) = 1 \,\}, \quad \text{cfg_ready}(u{+}1) = 1, \tag{8-3}$$

其中 $t$ 是总线接受写的拍，$u - t$ 是表写等待，上界 64 拍（第 3.7 节）。

## 8.5 复位与初始化

复位与清表过程见第 3.4 节。对配置模块而言：复位后寄存器回到默认值，27 个计数器清零；`tx_ready` 为 0 期间保持寄存器清空、`cfg_ready` 保持 0，配置写不被接受，读照常进行。

## 8.6 统计计数器

27 个计数器均为 32 位，计满回 0，只有复位清零。在 $t$ 拍发出读请求，$t + 2$ 拍返回对应统计脉冲在 $t - 1$ 拍及以前出现的次数。计数器分为：

- 发送（**7** 个）：收下的消息数；因 VL 未启用、队列满、超长、无空闲页而丢弃的消息数；A、B 网发出帧数。

- 接收，每网 **8** 个：完好帧、FCS 错、帧长错、帧头错、未知 VL、配置不允许、完整性检查丢弃、溢出。
- 接收，两网合计 **2** 个：冗余管理丢弃、交付。
- 保留 **2** 个：a_gdrop、b_gdrop，地址保留，恒为 0。

这一取样定义来自读通路：第 1 拍寄存的是本拍更新之前的计数值。

## 8.7 异常处理汇总

表 8-3 汇总全设计对各类异常的检测位置、处理方式和统计。异常消息按表中路径处理；发送侧拒收时经有界暂停完成回收，再消费并丢弃剩余字节，不会让应用无限等待。

表 8-3 异常处理汇总

| 异常 | 检测位置 | 处理 | 统计 |
|---|---|---|---|
| *发送* | | | |
| VL 或两网均未启用 | `tx_engine` | 拒绝入队，退回已用页 | drop_dis |
| 队列已满 | `tx_engine` | 同上 | drop_full |
| 超过 `max_payload` 或 1471 字节 | `tx_engine`、`tx_buf` | 同上；未收完的字节继续收下丢弃 | drop_long |
| 无空闲页 | `tx_buf` | 同上 | drop_nopage |
| MAC 欠载（按设计不发生） | `mac_tx` | 发不取反的 FCS 使对端必判错，吞掉本帧剩余字节 | 不计 tx_a/tx_b |
| *接收（每网独立）* | | | |
| 前导错、帧长越界 | `mac_rx` | 整帧丢弃 | 帧长错 |
| FCS 错 | `mac_rx` | 整帧丢弃 | FCS 错 |
| 固定字段、长度、分片或校验和错 | `rx_parse` | 试写回退 | 帧头错 |
| 环形缓冲放不下 | `rx_parse` | 本帧不写 | 溢出 |
| VLID 未映射 | `rx_engine` | 试写回退 | 未知 VL |
| VL 或本网未启用、超过 `max_payload` | `rx_engine` | 试写回退 | 配置不允许 |
| SN 不连续 | `rx_engine` | 试写回退，PSN 仍更新为本次 SN | 完整性检查丢弃 |
| 另一网已交付的副本 | `rx_engine` | 试写回退 | rm_drop |
| 交付队列已满 | `rx_engine` | 试写回退 | 溢出 |
| 清表完成前开始的帧 | `rx_parse` | 整帧忽略 | 不计 |
| *服务与配置* | | | |
| SNMP community 不符 | `afdx_snmp` | 不回应；Trap 启用时发 authenticationFailure | `BAD_COMMUNITY` |
| SNMP 请求超过 256 字节或代理忙 | `afdx_snmp` | 认领后丢弃，由管理端超时重发 | 忙时计入 `REQUESTS` |

接下页

表 8-3 （续）

| 异常 | 检测位置 | 处理 | 统计 |
|---|---|---|---|
| 615A 数据/文件检查错、状态传输失败、请求错误 | afdx_615a | 数据/文件错：ERROR 及失败状态文件；状态传输失败（含重试耗尽）：直接结束；名称/模式错：仅 ERROR；LCI/LNO：初始化拒绝 | STATUS 原因码 1–5（第 7.4 节） |
| SNTP 回应版本、模式或 Originate 不符 | afdx_rtc | 丢弃，下一周期重发请求 | — |
| 清表期间写配置 | afdx_cfg | cfg_ready= 0，不接受 | — |
| 未映射的地址 | afdx_cfg | 写无效果，读出 0 | — |

## 8.8 主机编程流程

主机经配置总线按下列顺序使端系统进入工作状态。每次表写都要等 cfg_ready 为 1（第 4 章）。

1. 初始化。释放复位后轮询 STATUS，等位 0（tx_ready）为 1，约 524 µs。
2. 端系统标识。写 USER_ID。
3. 发送 **VL**。对每条发送 VL 先写字 1–5（BAG、VLID、IP、默认端口），最后写字 0，同时置 en、网络选择、qmax 和 max_payload。
4. 接收 **VL**。对每条接收 VL 先写 rx_cfg[idx]，再写 rx_map[VLID]= $\{1, \text{idx}\}$，使映射生效时配置已经就绪。
5. 服务。为每项服务配置一条发送 VL（目的 IP、端口与服务端口一致），写服务的 TX_VL、端口和 community 等寄存器，最后写 ENABLE。
6. 开始收发。应用开始提交消息。VL 较多时按第 3.7.3 节错开各 VL 首条消息的提交时刻。

运行中修改配置。tx_cfg 的 en 只在入队时检查，已排队的消息授权时不再检查，也不清队列；授权会逐拍读取字 0–5。因此修改一条发送 VL 时，主机应先停止向它提交消息，把字 0 的 en 写 0，此步只改 en，其余字段保持不变；待该 VL 已提交的消息全部发出后再改其余字，最后重新启用。cfg_ok 只保证配置写不与表读撞在同一地址，不保证多字配置的原子更新。接收侧的 rx_cfg 和 rx_map 均保证单字写原子性；两表的组合更新不具事务原子性，也不会自动清除历史状态。

应用与服务配置。

- 应用须等待收发核心联合就绪的 tx_ready 再提交消息；app_tx_ready 不受该联合就绪信号门控。
- 各服务的 UDP 端口须互不冲突；RTC 模式和时间，以及 SNMP 的 ENABLE、community、TX_VL，仅在对应服务消息不在途时修改。

- 615A 忙时忽略启用、发送 VL、请求端口、超时、重传上限和文件版本的控制写；写 `0xB4` 后至少等 4 拍，仅在非忙时读 `0xB5`。

## 8.9 配置约束

硬件不检查配置之间的组合关系，下列约束由主机保证。违反时设计仍按规则丢弃并计数，不会挂死，但相应指标不再成立。

- 带宽。A、B 两网由同一个 MAC 状态机同拍驱动，只发 A 网或只发 B 网的帧同样占用一个发送时隙，所以约束针对全部发送 VL，不分网络：$\sum_v C(L_v)/\mathrm{BAG}_v \le 1$，其中 $L_v$ 为 `max_payload`，$C$ 为第 3.7 节的占线拍数，服务 VL 计入求和。超出时帧串行发出，实际间隔大于 BAG。
- 抖动。BAG 取 $2^k$ ms；错开各 VL 首条消息的提交时刻，使任一 VL 前面可能排着的帧按式 (3-3) 不超过 5828 拍（50 µs）。
- 缓存。页在组帧器读完后才归还，而 `q_count` 在授权时就减 1；正在接收的消息也在入队判决之前就已取页。所以除各队列中的消息外，还要计入组帧器持有的两帧（下一帧与正在发送的帧）和 `tx_buf` 正在接收的一条消息，各至多 3 页：$\sum_v \mathrm{qmax}_v \times \lceil L_v/512 \rceil + 9 \le 256$ 时不会因无空闲页丢弃消息；超出时按 drop_nopage 计数。
- 交付。交付口每拍一字节。一条 VL 关闭冗余管理而两网都接收时，每条消息交付两次。所有接收 VL 交付的载荷总率须低于 1 字节每拍；两网独立满速且都要交付时不满足，超出部分按溢出计数。
- 取值范围。`bag` $\le 2^{30}$ 拍；`qmax` $\le 63$；`max_payload` 为 1–1471；`skew_max` 小于老化门限 `0x4000` 个刻度（12 位字段自然满足）。

# 第 9 章　时序与资源

## 9.1　时序设计规则

为使 4096 条 VL 的查表通路和逐字节数据通路都能在 8 ns 内完成，全部 RTL 遵守表 9-1 的十一条规则。规则在编码前确定，并在每次综合后用时序报告复核。

表 9-1　时序设计规则

| 编号 | 规则 |
|---|---|
| 1 | 单时钟。 |
| 2 | 逐 VL 数据只放 RAM，不出现按 VL 数展开的触发器阵列或并行比较。 |
| 3 | 所有 RAM 只通过 `afdx_ram_sdp` 例化，地址、写数据和使能直接来自寄存器。小寄存器组注明用途（载荷 FIFO 8 项、归还 FIFO 4 项、接收描述 32 项）。 |
| 4 | RAM 读出先进寄存器，再做加、减、比较；中间最多一级不超过 4 选 1 的选择。综合估算：RAM 读出到寄存器 2.7 ns，直接接 32 位减法则为 4.4 ns。 |
| 5 | 一拍最多一级 32 位算术；不用乘法器和除法器，地址用位拼接。 |
| 6 | 多于 8 选 1 的多路选择分两拍。 |
| 7 | 数据逐拍推进，不在两字节之间插入额外处理周期；RAM 和算术用寄存流水，帧头期间预读。 |
| 8 | 同一拍读写同一地址的结果视为不确定，设计不依赖它。 |
| 9 | 模块输出以寄存器为主；组合例外为 ready 握手、`cfg_ok`，以及 `afdx_svc` 的数据选择与接收分发直通，后者只做选择和门控。 |
| 10 | 单一 125 MHz、8 ns 约束下综合 setup 裕量不小于 2.0 ns，不加 false path 或多周期例外。 |
| 11 | 综合报告的 RAM 写数据 hold 值逐项记录并分析来源。 |

规则 2 和 4 是“表加单主状态机”结构的基础：逐 VL 状态全部在 RAM 中，每张表只由一个状态机按固定流水访问，每拍的逻辑深度与 VL 数量无关。规则 5 使 RTC 的时间戳换算和偏差计算也拆成每拍一级加减，因此三个服务模块的路径深度与收发核心处于同一量级。

## 9.2　存储预算

全部存储都由 `afdx_ram_sdp` 例化，按声明的深度和位宽计算逻辑存储位数，见表 9-2。综合工具把这些逻辑 RAM 映射到器件的存储块，报告共用 340/1200 个（28.33%）。

`tx_payload` 和 `rx_map` 两张深表占去四分之三。

## 9.3　综合结果

综合使用 eLinx 3.0.7，器件 EQ6HL130、CSG484_H 封装，约束为单一 125 MHz 时钟，资源见表 9-3。

表 9-2　存储预算（逻辑位数）

| 表 | 深度 × 位宽 | 位数 | 说明 |
|---|---|---|---|
| tx_payload | 131072×8 | 1048576 | 256 页 × 512 字节 |
| tx_cfg | 1024×32 | 32768 | 128 条 VL × 8 字 |
| tx_desc | 256×44 | 11264 | 每页一项 |
| tx_state | 128×63 | 8064 | 每条 VL 一项 |
| tx_mnext/pnext/free | 3×256×8 | 6144 | 链表与空闲页 FIFO |
| rx_map | 65536×13 | 851968 | 任意 VLID 直接映射 |
| rx_state | 4096×44 | 180224 | 每条 VL 一项 |
| rx_cfg | 4096×32 | 131072 | 每条 VL 一字 |
| rx_ring_a/b | 2×4096×8 | 65536 | 每网 4 KiB |
| SNMP 请求、回应 | 2×256×8 | 4096 | |
| 615A 五块 | — | 102400 | 收发报文各 1 KiB，LUR 512 B，LUH 2 KiB，加载区 8 KiB |
| 合计 | — | 2442112 | 发送 1106816，接收 1228800，服务 106496 |

表 9-3　综合资源（综合后估算）

| 资源 | 用量 | 器件总量 | 占比 |
|---|---|---|---|
| LUT | 32847 | 136160 | 24.12% |
| FF | 13869 | 136160 | 10.19% |
| CLB | 4620 | 13616 | 33.93% |
| 存储块 | 340 | 1200 | 28.33% |
| DSP / PLL | 0 / 0 | 48 / 8 | 0% |
| 用户引脚 | 259 | 339 | 76.40% |

时序方面，8 ns 约束下 setup 裕量为 +2.522 ns，估算最高频率 182.548 MHz，满足规则 10 的 2.0 ns 裕量要求。

表 9-4 是主要模块的资源分布（综合层次报告，含子模块）。收发核心承担了 4096 条接收 VL 和 128 条发送 VL 的全部数据通路，逻辑单元合计约 14.5k；其中接收引擎和发送引擎分别只有 1127 和 1790 个逻辑单元，体现了逐 VL 状态放 RAM、引擎只保留一份流水的结构。

## 9.4　关键路径

按最小 setup 裕量选出的最慢路径见表 9-5。最慢路径在服务模块的报文解析状态机中，而不在线速收发通路上；收发核心的路径由规则 2–6 限定在 RAM 输出寄存、一级 32 位算术和分拍选择之内。

表 9-4　主要模块资源（综合后估算，含子模块）

| 模块 | 实例数 | 逻辑单元 | FF | 存储位数 |
|---|---|---|---|---|
| afdx_tx_core | 1 | 5677 | 2975 | 1106816 |
| afdx_tx_engine | 1 | 1790 | 1029 | 0 |
| afdx_tx_builder | 1 | 2091 | 962 | 0 |
| afdx_mac_tx | 1 | 597 | 155 | 0 |
| afdx_rx_core | 1 | 8814 | 4964 | 1208320 |
| afdx_rx_parse | 2 | 2466 | 1220 | 0 |
| afdx_rx_engine | 1 | 1127 | 489 | 0 |
| afdx_rx_deliver | 1 | 1975 | 481 | 0 |
| afdx_mac_rx | 2 | 371 | 222 | 0 |
| afdx_rtc | 1 | 4573 | 2190 | 0 |
| afdx_snmp | 1 | 4463 | 1261 | 4096 |
| afdx_615a | 1 | 7171 | 1358 | 102400 |
| afdx_cfg | 1 | 1954 | 1076 | 0 |
| afdx_svc | 1 | 120 | 7 | 0 |

表 9-5　综合后最慢路径

| 模块与起点 → 终点 | 逻辑级数 | 逻辑延迟 (ns) | setup 裕量 (ns) |
|---|---|---|---|
| afdx_615a: name_pos[1] → packet_type[1] | 15 | 5.403 | +2.522 |

RAM 写数据寄存器输入端的 hold 裕量为 −0.068 ns，在零走线延迟且时钟到 RAM 比到触发器晚 0.754 ns 的固定时钟偏斜模型下，估算增加约 0.07 ns 数据路径延迟即可满足 hold，但这不是布线保证，当前尚无布线后时序签核。

# 第 10 章　可验证性设计

本章说明设计为验证提供的观察手段。验证平台、用例、核对项、测量口径和结果见测试报告。

## 10.1　设计原则

设计让每条消息和每一帧的去向都能只从顶层端口确定，从而满足三点：

- 只看端口。系统级验证不需要探测内部信号，同一套激励可以直接用于 RTL 和综合后网表。
- 期望值可独立生成。帧格式、CRC、校验和、BAG 间隔和 IC/RM 判决都按协议规则定义，参考模型可以不读 RTL 独立算出期望结果。
- 逐条核对。每条消息、每一帧恰好对应一个结果脉冲，验证可以逐条比对去向，而不只是比较总数。

## 10.2　可观察性

- 事件端口。`ev_tx`、`ev_a`、`ev_b`、`ev_rm_drop`、`ev_deliver` 对每条发送消息给出 accept 或四种丢弃之一，对线上每一帧在其所在网络给出一个类别脉冲，完好帧再给出一个结果脉冲（第 6.6 节）。脉冲出现的时间有确定上界：发送消息的脉冲在末字节后 96 拍内，接收类别脉冲在帧尾后 32 拍内，结果脉冲在帧尾后 150 拍内。
- 计数器。27 个统计计数器经配置口读回，读值的取样时刻有精确定义（$t$ 拍发出读请求，$t+2$ 拍返回 $t-1$ 拍及以前的脉冲数，第 8 章），可以与事件记录逐拍对齐。
- 线路字节。GMII 输出直接来自 MAC 的输出寄存器，帧的每个字节和起始拍都在端口上可见；两网同拍起始，便于对照两网的输出。
- 服务状态。RTC 的偏差和校时次数、SNMP 的请求和 community 错误计数、615A 的状态、原因码和加载区内容都可经配置口读回（附录 B）。
- **RAM** 碰撞监视。`afdx_ram_sdp` 在仿真中检查同拍读写同一地址，出现即报错；该监视用 `` `ifndef SYNTHESIS `` 包住，不影响综合网表。它直接检验第 8.4 节“避让”设计是否成立。

## 10.3　可控性

链路上的各类异常都可以从端口构造：在 GMII 输入上改动字节、丢弃或重放整帧，即可产生 FCS 错、帧头错、序号乱序、双网时间差等情况；配置口可以随时改写 VL 配置以构造配置不允许、未知 VL 和队列满。设计不需要专门的测试模式。

# 第 11 章　设计边界与后续工作

## 11.1　设计边界

协议与容量边界。

- AFDX/IP：子 VL、IP 分片与重组、采样端口与队列端口语义、ICMP；发送的 UDP 校验和为 0，接收不按目的 IP 过滤。
- SNMP：未实现 GetBulk、单请求多个变量绑定、v1 格式 Trap，noSuchInstance 语义尚不完整；私有对象使用演示用命名空间。
- 615A：未实现信息操作和操作员定义卸载的后续传输、FIND、WAIT、ABORT、整文件重试、多文件加载，以及第 3 版新增 TFTP 选项的协商；TFTP ERROR 不核对来源端口，截断请求的长度检查尚不完整；LUH 不核对件号和硬件 ID，不处理支持文件、用户数据、扩展点、Load CRC-32、解压和解密。
- RTC：不检查服务器 stratum 和全零时间戳。
- 上板接口：接实体 PHY 所需的 RX_CLK 弹性缓冲和 GTX_CLK 输出（第 3.4 节）；外部复位同步器。
- 容量与调度：页池可驻留 85 条最长消息；跨 VL 不保证全局入队顺序，提前候选可使小帧等待；双网非冗余满速输入可填满环或 DQ，溢出时计数；到达间隔超过 `skew_max` 的副本可能再次交付。

指标的适用条件。抖动和时延的设计值以第 8.9 节的配置约束为前提：带宽不超过线速、任一 VL 前面可能排着的帧按式 (3-3) 不超过 5828 拍、交付总率低于每拍一字节。配置之间的组合关系由主机保证，硬件不检查。

验证范围。本设计的功能验证基于 RTL 和综合后网表仿真（第 10 章）；综合时序为未布线估算。尚未完成完整性能矩阵的网表仿真，布局布线网表仍有 RAM 位序与配套模型不一致的问题；本设计未进行布局布线时序签核和实体硬件测量，RTC 未在独立晶振和非对称链路下验证。

## 11.2　后续工作

1. 完成布局布线，针对服务模块中的长组合路径（表 9-5）做流水切分，完成 125 MHz 时序收敛。
2. 上板测试 GMII 接口，在实体网络上测量吞吐、时延和对时误差。
3. 在 `tx_cfg` 保留字中增加每条 VL 的初始相位，由硬件错开同时启用的 VL，取代主机错开提交时刻。
4. 补充 SNMP GetBulk、615A 多文件加载等协议功能。

# 附录 A　源文件与默认参数

## A.1　源文件

RTL 共 21 个 Verilog 文件，均在仓库 `rtl/` 目录。

表 A-1　RTL 源文件

| 文件 | 作用 |
|---|---|
| `AFDX_top.v` | 顶层：双网收发核心、服务通道、三项服务、配置与时间基准 |
| `afdx_timebase.v` | 周期计数 `now` 与粗计时 `now_c` |
| `afdx_cfg.v` | 地址译码、寄存器、27 个计数器、配置表写握手 |
| `afdx_svc.v` | 应用与服务的消息轮询发送、按端口分发接收 |
| `afdx_tx_core.v` | 发送核心外壳，例化发送侧各模块和 7 张表 |
| `afdx_tx_buf.v` | 消息分页写入、入队请求、失败退页 |
| `afdx_tx_free.v` | 空闲页 FIFO |
| `afdx_tx_engine.v` | BAG 调度：扫描、入队、授权、起始确认 |
| `afdx_tx_builder.v` | 组帧、IP 头校验和、载荷预读、页归还 |
| `afdx_mac_tx.v` | 双网发送 MAC：前导、FCS、帧间隔、欠载保底 |
| `afdx_crc32_d8.v` | 逐字节 CRC32 组合逻辑 |
| `afdx_rx_core.v` | 接收核心外壳，例化接收侧各模块和 5 张表 |
| `afdx_mac_rx.v` | 接收 MAC：找帧起始符、CRC 校验、剥离 FCS |
| `afdx_rx_parse.v` | 帧头检查、试写环形缓冲、定类别 |
| `afdx_rx_engine.v` | VL 查表、完整性检查、冗余管理、老化 |
| `afdx_rx_deliver.v` | 从环形缓冲逐字节交付 |
| `afdx_ram_sdp.v` | 简单双口同步 RAM，全设计唯一的 RAM 例化方式 |
| `afdx_rtc.v` | 实时时钟与 SNTP 主从对时 |
| `afdx_ntp_fraction.v` | 纳秒与 NTP 小数的逐位换算 |
| `afdx_snmp.v` | SNMP 代理：BER 编解码、MIB、Set、Trap |
| `afdx_615a.v` | 615A 目标端：TFTP、LUH/数据文件检查、上传与卸载 |

## A.2　默认参数

表 A-2　顶层参数与默认容量

| 参数 | 默认值 | 含义 |
|---|---|---|
| `TX_PAGE_AW` | 8 | 发送缓存 256 页 |
| `TX_OFF_AW` | 9 | 每页 512 字节，共 128 KiB |
| `LOOKAHEAD` | 512 | 提前选择窗口，512 拍（4.096 μs） |

接下页

表 A-2　（续）

| 参数 | 默认值 | 含义 |
|---|---|---|
| `RX_RING_AW` | 12 | 每网接收环形缓冲 4 KiB |
| `RX_DQ_AW` | 5 | 交付描述队列 32 项 |
| 发送 / 接收 VL | 128 / 4096 | 接收映射表 65536 项 |
| 帧间隔 | 12 拍 | 96 ns |
| 初始化 | 65538 拍 | 解除复位到 `tx_ready`，524.304 µs |
| SNMP 请求缓存 | 256 B | 单变量绑定 |
| 615A 加载区 | 8 KiB | 头文件区 2 KiB，LUR 区 512 B |
| 615A 超时 / 重传 | 2 s / 1 次 | 可经寄存器修改 |
| RTC 同步周期 | 125000 拍 | 1 ms，可经寄存器修改 |

# 附录 B　服务寄存器

基本寄存器、计数器和配置表地址见表 8-1，服务寄存器见表 B-1。RW 为可读写，RO 为只读。

表 B-1　服务寄存器

| 地址 | 名称 | 读写 | 含义 |
|---|---|---|---|
| *RTC* | | | |
| 0x80 | CTRL | RW | 位 0 启用，位 1 主端；写位 2 装入暂存时间 |
| 0x81 | PERIOD | RW | 从端请求间隔（拍），复位 125000 |
| 0x82 | TX_VL | RW | 发送 VL，低 7 位 |
| 0x83 | UDP_PORT | RW | UDP 端口，复位 123 |
| 0x84 | SEC | RW | 写秒暂存值；读当前秒并快照纳秒 |
| 0x85 | NS | RW | 写纳秒暂存值；读快照纳秒 |
| 0x86 | OFFSET_SEC | RO | 最近偏差的有符号秒部分 |
| 0x87 | OFFSET_NS | RO | 最近偏差的纳秒余数 |
| 0x88 | SYNC_COUNT | RO | 从端校时次数 |
| *SNMP* | | | |
| 0x90 | ENABLE | RW | 位 0 启用代理 |
| 0x91 | TX_VL | RW | 回应与 Trap 的发送 VL |
| 0x92 | COMMUNITY_LENGTH | RW | 1–16，复位 6 |
| 0x93–0x96 | COMMUNITY_0–3 | RW | community 内容，复位为 public |
| 0x97 | SAMPLE_CYCLE | RO | 上次 MIB 读取时的 now |
| 0x98 | UPTIME | RO | sysUpTime，单位 10 ms |
| 0x99 | REQUESTS | RO | 被认领的请求数 |
| 0x9A | BAD_COMMUNITY | RO | community 不符次数 |
| 0x9B | TRAP_ENABLE | RW | 位 0 启用 Trap |
| 0x9C | TRAPS_SENT | RO | 已发送 Trap 数 |
| 0xA0–0xA3 | DESCRIPTION_0–3 | RO | sysDescr 字符串 |
| *615A* | | | |
| 0xB0 | ENABLE | RW | 位 0 启用 |
| 0xB1 | TX_VL | RW | 发送 VL |
| 0xB2 | STATUS | RO | 位 1:0 空闲/进行中/成功/失败；位 15:8 原因码 |
| 0xB3 | LOADED_BYTES | RO | 本次已写字节数 |
| 0xB4 | MEMORY_ADDRESS | RW | 加载区读回地址，低 13 位 |
| 0xB5 | MEMORY_DATA | RO | 加载区读回字节 |
| 0xB6 | LUS_COUNTER | RO | 本次状态文件数 |
| 0xB7 | RETRY_TOTAL | RO | 累计超时重传次数 |

接下页

表 B-1 （续）

| 地址 | 名称 | 读写 | 含义 |
|---|---|---|---|
| 0xB8 | REQUEST_PORT | RW | TFTP 请求端口，复位 59 |
| 0xB9 | TIMEOUT_CYCLES | RW | 超时拍数，复位 250000000 |
| 0xBA | RETRY_LIMIT | RW | 重传上限，复位 1 |
| 0xBB | PROTOCOL_VERSION | RW | A4 或 A3 |

# 附录 C　内部接口信号

本附录逐信号定义第 4.7 节汇总的内部接口。

## C.1　消息接口

服务通道 afdx_svc 把选中的消息以字节流交给发送核心，见表 C-1。没有服务报文时这组信号与应用口组合直通。

表 C-1　消息接口（afdx_svc → afdx_tx_buf）

| 信号 | 位宽 | 含义 |
|---|---|---|
| m_valid、m_ready | 1、1 | 字节握手；m_ready 只在 tx_buf 处于 IDLE、RECV、DRAIN 时为 1 |
| m_data、m_last | 8、1 | 消息字节、末字节标记 |
| m_vl | 7 | 发送 VL 序号，首字节取样 |
| m_port_en、m_sport、m_dport | 1、16、16 | 为 1 时用消息自带 UDP 端口；首字节取样 |

## C.2　入队与授权

afdx_tx_buf 写完消息后把描述写入 tx_desc，再向引擎发入队请求；引擎选中一条消息后把整帧字段一次交给组帧器，组帧器在帧真正起始时回报起始时刻，见表 C-2。

表 C-2　入队、授权和起始回报接口

| 信号 | 位宽 | 含义 |
|---|---|---|
| 入队（*tx_buf* → *tx_engine*，请求应答） | | |
| enq_valid | 1 | 保持到 enq_ack |
| enq_vl、enq_head、enq_len | 7、8、11 | VL 序号、消息首页号、字节数 |
| enq_ack、enq_result | 1、3 | 单拍应答；0 收下，1 VL 未启用，2 队列满，3 超长（4 无空闲页由 tx_buf 自行判定） |
| tx_desc[43:0] | 44 | {len[10:0], port_en, sport[15:0], dport[15:0]}，以首页号为地址 |
| 授权（*tx_engine* → *tx_builder*，单槽交接） | | |
| grant_valid、grant_ready | 1、1 | grant_ready = ¬pending，即“下一帧”位置空闲 |
| grant_a、grant_b | 1、1 | 两网启用位（配置字 0 的 [1]、[2]） |
| grant_timed、grant_time | 1、32 | 为 1 时帧不得早于 grant_time 起始；为 0 表示立即 |
| grant_vlid | 16 | 配置字 2 |

接下页

表 C-2 （续）

| 信号 | 位宽 | 含义 |
|---|---|---|
| grant_src、grant_dst | 32、32 | 配置字 3、4 |
| grant_sport、grant_dport | 16、16 | port_en 为 1 取消息自带端口，否则取配置字 5 |
| grant_len、grant_sn、grant_page | 11、8、8 | 载荷字节数、帧序号、首页号 |
| *起始回报（tx_builder → tx_engine）* | | |
| start_valid、t_start | 1、32 | 单拍脉冲；t_start 为首个前导字节出现那一拍的 now |

## C.3 组帧器与 MAC

组帧器以字节流把帧内容（目的 MAC 到 SN，不含前导和 FCS）交给 MAC，MAC 决定何时取走，见表 C-3。

表 C-3 帧字节接口（tx_builder ↔ mac_tx）

| 信号 | 位宽 | 含义 |
|---|---|---|
| s_valid、s_ready | 1、1 | 字节握手；s_ready 在 byte_en= 1 且满足以下任一条件时为 1：前导末拍；DATA 且 last_byte= 0；DRAIN |
| s_data_a、s_data_b | 8、8 | 两网字节，仅第 11 字节不同 |
| s_last | 1 | 与 SN 字节同拍 |
| s_en_a、s_en_b | 1、1 | 起始拍取样，选择本帧发往哪一网 |
| tx_sof、tx_eof | 1、1 | 首个前导字节、帧尾（FCS 发完）的单拍脉冲 |
| tx_underrun | 1 | 数据阶段上游未备好字节的保底告警 |

## C.4 帧描述与交付队列

每网解析器对完好帧给出一份 95 位帧描述，引擎取走后判决并回送提交或回退；通过的帧在交付队列中登记一项，见表 C-4。

表 C-4 帧描述与交付队列项

| 字段 | 位宽 | 含义 |
|---|---|---|
| *帧描述（rx_parse → rx_engine，单槽交接，高位在前）* | | |
| vlid | 16 | 目的 MAC 第 4–5 字节 |
| sn | 8 | 帧末 SN |
| sport、dport | 16、16 | UDP 端口 |
| n | 11 | 载荷字节数 |

接下页

表 C-4 （续）

| 字段 | 位宽 | 含义 |
|---|---|---|
| start | 12 | 本帧载荷在环形缓冲中的起点 |
| arrival | 16 | 帧尾时刻的 now_c |
| desc_valid、desc_take | 1、1 | 交接握手 |
| resolve_valid、resolve_commit | 1、1 | 引擎判决：提交则 wr_commit 前进到本帧末，否则 wr_trial 退回 |
| *交付队列项（rx_engine → rx_deliver，32 项寄存器）* | | |
| net | 1 | 0 为 A 网，1 为 B 网 |
| start、n | 12、11 | 环形缓冲起点和字节数 |
| idx | 12 | 接收 VL 序号 |
| sport、dport | 16、16 | UDP 端口 |

交付口 d_* 的字段与应用接收口相同。服务通道把它同时广播给应用和三个服务，服务在首字节同拍以组合信号 svc_rx_claim 认领；认领结果寄存到末字节，被认领的消息不出现在应用口。

## C.5 服务发送总线

三个服务的发送口打包成一组总线接到 afdx_svc，按 RTC、SNMP、615A 从低位到高位拼接，见表 C-5。各服务内部用同一套字节流握手驱动自己的那一段。

表 C-5 服务发送总线 svc_tx_*

| 信号 | 位宽 | 拼接 |
|---|---|---|
| svc_tx_valid、_ready、_last、_port_en | 各 $3\times1$ | [0] RTC，[1] SNMP，[2] 615A |
| svc_tx_data | $3\times8$ | [7:0] RTC，[15:8] SNMP，[23:16] 615A |
| svc_tx_vl | $3\times7$ | [6:0]、[13:7]、[20:14] |
| svc_tx_sport、svc_tx_dport | 各 $3\times16$ | [15:0]、[31:16]、[47:32] |

# 附录 D　术语与参考文献

## D.1　术语

表 D-1　本文专用术语

| 术语 | 含义 |
|---|---|
| 拍 | 一个 125 MHz 时钟周期，8 ns |
| R$n$ | 官方答疑《FPGA 创新大赛问题汇总》（1010 版）Sheet1 第 $n$ 行 |
| PSN | 同一网上同一 VL 上一帧的序列号 |
| 试写 / 提交 / 回退 | 载荷先写入环形缓冲但不生效；通过检查则生效，否则撤销 |
| 候选 | 调度引擎选出、等待授权给组帧器的 VL |
| `pend_vl` | 已授权但尚未开始发送的 VL |
| `cfg_ok` | 引擎在本拍不发出配置表读的指示 |
| 有效份 | 一条消息在某一网上的、未被链路动作改动且接收端允许接收的副本 |

## D.2　参考文献


[1] ARINC Specification 664 Part 7: Avionics Full Duplex Switched Ethernet Network, 2005-06-27.

[2] ARINC Report 615A-2: Software Data Loader Using Ethernet Interface, 2002-05-10.

[3] ARINC Report 665-1: Loadable Software Standards.

[4] RFC 1350: The TFTP Protocol (Revision 2), 1992.

[5] RFC 2347: TFTP Option Extension, 1998.

[6] RFC 1157: A Simple Network Management Protocol (SNMP), 1990.

[7] RFC 3416: Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP), 2002.

[8] RFC 3418: Management Information Base (MIB) for the Simple Network Management Protocol (SNMP), 2002.

[9] RFC 4330: Simple Network Time Protocol (SNTP) Version 4 for IPv4, IPv6 and OSI, 2006.

[10] IEEE Std 802.3: IEEE Standard for Ethernet.

[11] 中科亿海微. 全国大学生嵌入式芯片与系统设计竞赛 FPGA 创新设计赛道选题指南.